

FuzzyDedup: Secure Fuzzy Deduplication for Cloud Storage

Tao Jiang^{ID}, *Member, IEEE*, Xu Yuan^{ID}, *Senior Member, IEEE*, Yuan Chen, Ke Cheng^{ID},
Liangmin Wang^{ID}, *Member, IEEE*, Xiaofeng Chen^{ID}, and Jianfeng Ma^{ID}

Abstract—Data deduplication is of critical importance to reduce the storage cost for clients and to relieve the unnecessary storage pressure for cloud servers. While various techniques have been proposed for secure deduplication of identical files/blocks, the effective and secure deduplication solutions on fuzzy similar data (image, video, and others) which occupy a large portion in the real world across wide applications, remain open. In this article, we propose a novel deduplication system, named Fuzzy Deduplication (FuzzyDedup), to implement the secure deduplication of similar data (i.e., similar files, chunks, or blocks). In particular, we leverage the similarity-preserving hash, a fuzzy extractor based on error-correcting codes, and the encryption with customized design to construct a fuzzy-style deduplication encryption scheme (FuzzyMLE), achieving the ciphertext-based deduplication for similar data. Besides, to defend against data ownership cheating attack and duplicate-faking attack, a fuzzy-style proof of ownership scheme (FuzzyPoW) is designed for the cloud server to securely verify a client in possession of the similar data. To further enhance security and efficiency, we also propose both server-aided and random-tag FuzzyMLE to make FuzzyDedup robust against off-line brute-force attack and to support tag randomization, respectively. Then, we design Hamming distance reduction and tag cutting optimization algorithms to improve the tag query efficiency of FuzzyDedup. In the end, we formally prove the security of our solution and conduct experiments on real-world datasets for performance evaluation. Experimental results exhibit the efficiency of FuzzyDedup in terms of computation cost and communication overhead.

Index Terms—Cloud storage, security, fuzzy deduplication, message-locked encryption, proof of ownership

1 INTRODUCTION

THE huge amount of near-duplicate data, e.g., modified texts, images, and videos, abound in the real-world applications. For example, Facebook has about 0.25 billion images uploaded daily. [1] has revealed that in their crawled documents from the web, around 14%–52% documents from a single crawling trail and 0.7%–2.5% documents among

different crawling trails are near-duplicates. To reduce the wastage of costly storage, extensive solutions have been proposed to detect or eliminate fuzzy near-duplicate data, such as warehouses fuzzy duplicate texts elimination [2], [3], [4], near-duplicate images detection [5], [6], and similar/near-duplicate audios/videos elimination [7], [8], [9], etc, where users can tolerate a certain level of data modification/loss without impacting the data quality. Beyond storage costs saving, the elimination of fuzzy near-duplicates also saves network bandwidth and improves the quality of search indexes [8]. Many real-world softwares, such as Duplicate Cleaner [10], VisiPics [11], and dupeGuru [12], were also developed, to support such applications for providing the similar data deduplication functionality. Such evidences exhibit the necessity and importance of deduplicating fuzzy similar data. On the other hand, many individuals and enterprises have adopted cloud storage services, such as Amazon S3, Google Drive, Dropbox and others, to relieve their local storage burden [13]. This promotes the similar data deduplication across users during data outsourcing, which can substantially save storage costs for both standard file systems and back-up applications at the cloud side.

To date, various data deduplication mechanisms over cloud storage platforms [14], [15], [16] have been developed, but they can only deduplicate the identical data in the plaintext domain. Although some deduplication mechanisms have been proposed in the ciphertext domain, such as Convergent Encryption (CE) [17], Message-Locked Encryption (MLE) [18] and their variants [19], [20], [21], [22], where files can only be accessed by legitimate owners and resist against curious Cloud Storage Providers (CSPs), they can only

- Tao Jiang is with the State Key Laboratory of Integrated Service Networks, Xidian University, Xi'an 710071, China, and also with the State Key Laboratory of Integrated Service Networks, Guangxi Key Laboratory of Cryptography and Information Security, Xidian University, Xi'an 710071, China. E-mail: jiangt2009@gmail.com.
- Xu Yuan is with the School of Computing and Informatics, University of Louisiana at Lafayette, Lafayette, LA 70504 USA. E-mail: xu.yuan@louisiana.edu.
- Yuan Chen, Xiaofeng Chen, and Jianfeng Ma are with the State Key Laboratory of Integrated Service Networks, Xidian University, Xi'an 710071, China. E-mail: {yuanchen, xfchen}@xidian.edu.cn, jfma@mail.xidian.edu.cn.
- Ke Cheng is with the School of Computer Science and Technology, Xidian University, Xi'an 710071, China. E-mail: kechengstu@gmail.com.
- Liangmin Wang is with the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang 212013, China. E-mail: wanglm@ujs.edu.cn.

Manuscript received 17 Nov. 2021; revised 10 June 2022; accepted 17 June 2022.
Date of publication 22 June 2022; date of current version 13 May 2023.

This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFB0804103, in part by the Foundation for Innovative Research Groups of the National Natural Science Foundation of China under Grant 62121001, in part by the Leading-edge Technology Program of Jiangsu Natural Science Foundation under Grant BK20202001, in part by the Fundamental Research Funds for the Central Universities under Grant XJS211502, and in part by the Guangxi Key Laboratory of Cryptography and Information Security under Grant GCIS201716.

(Corresponding author: Jianfeng Ma.)

Digital Object Identifier no. 10.1109/TDSC.2022.3185313

perform secure deduplication of identical files (i.e., $F = F'$). Among these methods, the chunk-based solutions are appealing, which can deduplicate identical chunks of the similar files in both plaintext domain [23], [24] and ciphertext domain [25], [26]. For example, they can deduplicate chunk b_i of F and b'_j of F' when $F \neq F'$ but $b_i = b'_j$. However, the nature of these solutions is to deduplicate identical chunks while not similar ones, e.g., $b_i \approx b'_j$. Also, determining the suitable-sized chunk for obtaining identical chunks and achieving effective deduplication is extremely hard.

In this paper, we aim to develop a secure fuzzy deduplication scheme to securely deduplicate clients' outsourced fuzzy similar data, which can be similar files/blocks/chunks but may not be identical. Different from the identical data (i.e., identical files, chunks, or blocks) deduplication, the main challenge here lies in that a user shall be able to verify the plaintext similarity (i.e., distance) between his data and that of others in the encrypted domain without revealing sensitive information to the cloud server. In addition, this data similarity verification shall not rely on the assistance from any online data owner, since we cannot expect that they always stay online to offer help for performing the similarity computation. Although Li *et al.*'s scheme [27] has proposed the first secure cross-client similar image deduplication solution for cloud storage. However, they assume that clients belong to the same group with a trusted group manager, who is responsible for sharing encryption keys and helping users to deduplicate those similar images. Hence, it is not applicable to public cloud platforms for serving a large number of users, typically having no such a trust manager. Later, [28] and [29] explored the proof of ownership and user-aided secure fuzzy deduplication schemes, respectively. However, the former inherited the limitations of Li *et al.*'s scheme and the later required the assistance from an online data owner. Therefore, how to design a secure and practical fuzzy similar data deduplication across clients without the aid of a trust manager or online data owners remains an open and challenging problem.

To tackle the aforementioned challenges, we design a new deduplication system, named as FuzzyDedup, for secure and efficient similar data (i.e., similar files, chunks, or blocks) deduplication. Since traditional cryptographic tools such as cryptographic hash functions (e.g., SHA) and encryption algorithms (e.g., AES), cannot help to realize secure deduplication of similar data, we leverage a series of novel techniques such as similarity-preserving hash algorithm, fuzzy key extractor based on error-correcting code, encryption based on bit-wise eXclusive-OR (XOR), and secure Hamming distance computation protocol based on Oblivious Transfer (OT), with customized design to implement the effective fuzzy deduplication. Specifically, FuzzyDedup, belonging to the client(source)-side deduplication system, consists of two key components, i.e., Fuzzy Message-Locked Encryption scheme (FuzzyMLE) and Fuzzy Proof of oWnership scheme (FuzzyPoW), where the former one is used for users to conduct data encryption and the latter one offers the functionality of data ownership proof to the cloud server. More precisely, the cloud server can securely conduct deduplication of ciphertexts within certain distance range (i.e., Hamming distance) while reducing the data transfer overhead over the network from a client to the

server. Furthermore, we propose both server-aid and random-tag-based FuzzyMLE solutions to strengthen FuzzyDedup against data ownership cheating attack and duplicate-faking attack, by allowing the cloud server to verify whether a client is in possession of a similar file before the data deduplication is conducted. In the end, we provide the formal security proof and efficiency analysis for the designed FuzzyDedup system, while conducting experiments on real-world datasets to evaluate the performance of FuzzyDedup. Both the theoretical analysis and the systematic evaluation have demonstrated that, FuzzyDedup enables the cloud server to securely and efficiently deduplicate users' fuzzy similar data.

Our main contributions are summarized as follows:

- We propose a fuzzy deduplication system, named as FuzzyDedup, which can help to achieve secure client-side deduplication for users' fuzzy similar data when uploading to the public cloud, without relying on the central trusted manager. FuzzyDedup generalizes the functionalities of both deduplication encryption and data ownership proof by designing the respective fuzzy-style solutions, i.e., FuzzyMLE and FuzzyPoW, for similar data, and addressing the functionalities and security challenges that have not been covered so far in existing secure deduplication solutions.
- We leverage a series of prominent technologies, such as similarity-preserving hash, fuzzy extractor, oblivious transfer, with customized rectification in the design of our solution, to achieve secure data encryption and cross-client data similarity match. In addition, we extend our solution and construct various variants by employing server-aided key generation, tag randomization, random sampling and Hamming query optimization techniques, to further enhance the security of both FuzzyMLE and FuzzyPoW against the potential attacks while improving the efficiency of FuzzyDedup.
- We formally prove the security of FuzzyDedup and demonstrate, via experiments on real-world datasets, that it can provide security benefits for data outsourcing while retaining effective client-side similar data deduplication. Our optimization of tag similarity detection is demonstrated that an incoming file can be efficiently checked against all existing tags in the cloud storage.

2 MODELS

2.1 System Model

We consider a public cloud storage system consisting of a remote cloud server (i.e., S) and a set of clients (i.e., C_s). S accepts the registration from C_s and manages accounts for them to provide cloud storage services. To protect data confidentiality, C_s shall leverage some cryptographic tools to encrypt the sensitive data before outsourcing them to S . Assume that S offers a set of professional services, such as near-duplicate data cleaning, similar picture deletion, *etc.*, requiring the capability of performing efficient (client-side) deduplication operation (on the encryption domain) if

clients have similar data for uploading. This is popular in practice, where ciphertexts uploading from different clients may include a large number of similar files in their corresponding plaintexts, such as image, video data, *etc.* In these data applications, similar data deduplication will not impact the data quality. Hence, if S can perform the fuzzy deduplication over Cs' ciphertexts and ensure that only one copy of similar contents is uploaded and stored in its database, the storage overhead can be significantly reduced. Notably, an intuitive design of secure fuzzy deduplication is to apply the chunk-based method to divide each file into chunks and then apply MLE to each chunk of a file, such that if the encrypted chunks are identified, they will be deduplicated. This scheme is indeed feasible to conduct similar files deduplication in some cases. However, how to determine the suitable chunk size for cross-client ciphertext deduplication remains challenging: If the chunk size is too large, the deduplication ratio will be low; If too small, the excessive computation, storage, and management overhead will incur. The nature of this solution is to deduplicate the identical chunks rather than the similar one, which cannot be applied to our application scenarios. In addition, it cannot work well if the fuzzy bit-wise differences distribute across the entire file. Therefore, the secure fuzzy deduplication remains open, calling for a new solution.

We aim to design a secure deduplication system, named FuzzyDedup, which not only provides the basic operations for clients, such as Setup, Upload, Download and Delete, allowing Cs to setup the system for uploading, downloading, and deleting files to/from S 's database, respectively, but also offers two new functions: file encryption/decryption protocol (abbreviated as FuzzyMLE) and file ownership proof protocol (abbreviated as FuzzyPoW), to support similar data deduplication and ownership proof claiming. FuzzyMLE is executed between Cs and S , allowing Cs to encrypt their data to protect data secrecy before outsourcing to S , while supporting the deduplication operations on both identical and similar data within a certain distance, i.e., Hamming distance. Notably, the deduplication within this distance will not impact the quality of the original data, operating similarly as many plaintext deduplication systems [10], [11], [12]. In addition, FuzzyDedup will also allow Cs to be capable of claiming the ownership of their data to S . FuzzyPoW, as one of the core functions in FuzzyDedup, is designed to implement such a goal, allowing S to verify whether a client C is in possession of the specific data copy instead of only its hash tag. Also, the data possession attestation in FuzzyPoW is judged by similarity measurement, not only for simple data identical verification. Considering system efficiency in practical applications, the design of FuzzyDedup mainly relies on symmetric cryptographic technologies in our model, while data owners are not required to stay on-line to assist S or other clients to complete the data encryption and deduplication procedures.

2.2 Security Model

In FuzzyDedup, we assume there is no trusted entity to assign or manage the shared keys. Additionally, two adversaries are taken into account in our threat model.

An *honest-but-curious server* S will faithfully offer the designated secure deduplication and storage services, but may attempt to steal and learn the underlying content of the stored ciphertexts. More precisely, even there is no correct decryption key, S may try to break the secrecy property of the encryption scheme and crack the respective plaintext.

A *malicious client* C attempts to launch the *data ownership cheating attack* by using short fingerprints of files (e.g., hash tags) acquired via certain public channels. Also, it may perform the *duplicate-faking attack* or *data poison attack*, where C utilizes a correct tag for duplicate checking but uploading a fake ciphertext (that is not consistent with the tag) to compromise the file integrity of other clients.

Since no existing MLE achieves the semantic-style-security [18], an adversary, given a ciphertext computed with MLE, can recover the respective plaintext in $O(n)$ decryption trials if it is drawn from a message space of size n . Therefore, similar to the MLE which guarantees privacy under chosen distribution attacks, we consider the best possible security in FuzzyDedup and assume that the data for deduplication have high min-entropy. In other words, we only consider that our solution is applied to messages that are unpredictable and there is no trusted third party. FuzzyDedup provides secure client-side deduplication that can significantly reduce the cost incurred in communication and storage, where similar duplicate data will be deduplicated before arriving at S . However, it is difficult to prevent an attacker from identifying whether a specific file was previously uploaded to S through checking whether the deduplication occurs. To prevent such an attacker, one possible solution is that all clients' files have to be uploaded to S and the deduplication process is only run on S . The major drawback of such a solution is that it results in a communication inefficient *post-processing* deduplication.

2.3 Design Goals

In the design of the FuzzyDedup system, we have the following goals in terms of *security* and *efficiency*.

Security Goals consist of *correctness* and *privacy* requirements. The *correctness* requires that both the FuzzyMLE and FuzzyPoW are correct. First, for any two clients having similar plaintexts (e.g., w and w'), if they want to store their ciphertexts (i.e., c_w and $c_{w'}$) to S , S can detect the similarity by using FuzzyMLE and only store one copy (w or w'). When one client C from the two downloads the deduplicated ciphertext from S , she/he can decrypt the ciphertext with the secret key stored locally. Second, during the data ownership proof, the output of the party (i.e., the Hamming distance output by S) must be consistent with the actual input from C and S . Since FuzzyPoW can prevent a malicious client from realizing *duplicate-faking attack* [18], we do not need to consider the tag correctness problem in our system. On the other hand, the *privacy* of FuzzyMLE means the best possible privacy. That is, when messages are unpredictable (i.e., have high min-entropy) and the similarity-preserving hash function has both digest similarity and one-way properties, it is difficult for an adversary (including S) to obtain information of plaintexts. The *privacy* of FuzzyPoW means that all information, that an adversary

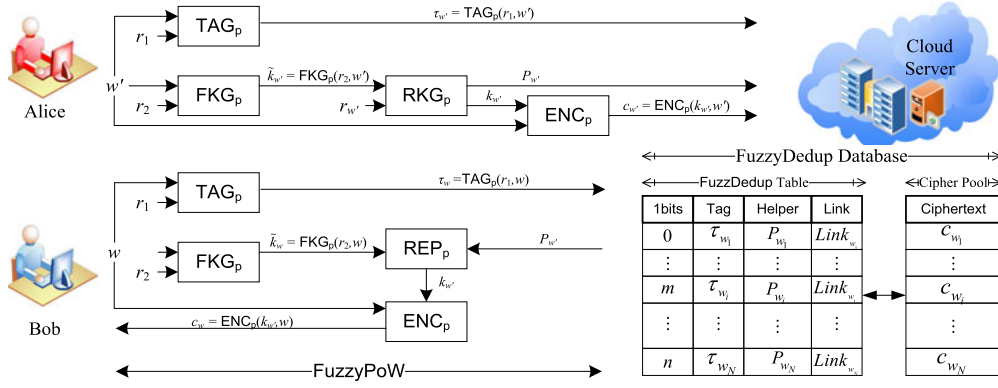


Fig. 1. Fuzzy deduplication system. r_1 and r_2 are two random public parameters, which make $\tilde{k}_{w'}$ independent of the tag $\tau_{w'}$. $r_{w'}$ is randomly chosen as an input to $[n, k, 2t + 1]_x$ error-correcting code. p represents some public parameters. $\text{dis}_{\text{Ham}}(w, w') < T$ and $\text{dis}_{\text{Ham}}(k_w, k_{w'}) < t$, where T and t are Hamming distance thresholds between files and their corresponding fuzzy keys.

learns, is only from the corresponding output alone. So, S obtains nothing about C_s ' input from the output.

Efficiency Goals include computation, storage, and communication efficiency. *Computation efficiency* means that FuzzyDedup can help S deduplicate similar data without involving heavy computation costs. *Storage efficiency* requires that, for the data within a specific Hamming distance, only one copy of C_s ' ciphertexts is stored in S . *Communication efficiency* requires that C_s do not need to upload similar ciphertexts to S and the communication overhead is less than the data size in the long run.

3 FUZZY DEDUPLICATION SYSTEM

3.1 Notations

Let x be a binary string over $\{0, 1\}$ and denote $|x|$ as its bit length, $x[i]$ as the i th bit and $x[i, j] = x[i] \dots x[j]$ for $1 \leq i \leq j \leq |x|$. Given a bit b , $\bar{b}$ represents the logical negation on b . If X is a finite set, then $|X|$ represents the number of its elements and $x \leftarrow \$X$ means we uniformly take an element from X and assign it to x . If X is a discrete random variable, we denote P_X as the probability distribution of X . We let U_ℓ be the uniform distribution $\{0, 1\}^\ell$ and use $X \stackrel{c}{=} Y$ to denote the computational indistinguishability between two distributions of X and Y . Assume $\lambda \in \mathbb{N}$ is a security parameter, then 1^λ represents its unary representation. If an algorithm f is randomized, then $f(r, x)$ denotes the output of f with the input of x and a random value r . For an adversary $\mathcal{A}$, the predictability of a random variable X is $\max_a \Pr[X = x]$, and the min-entropy $H_\infty(X)$ is $-\log(\max_a \Pr[X = x])$. A random variable with min-entropy of at least m is called an m -source. In this paper, a two-party interactive protocol is formally defined as follows:

$$\Pi : [\mathcal{P}_1 : in_1; \mathcal{P}_2 : in_2] \rightarrow [\mathcal{P}_1 : out_1; \mathcal{P}_2 : out_2],$$

where $\mathcal{P}_1$ and $\mathcal{P}_2$ represent two parties, in_i and out_i denote the input and output of $\mathcal{P}_i$ ($i \in \{1, 2\}$), respectively.

3.2 System Overview

FuzzyDedup consists of two key components, i.e., FuzzyMLE and FuzzyPoW. The function of FuzzyMLE is to offer an effective scheme for C_s to encrypt their files before outsourcing them to S , while allowing C and S to conduct

client-side deduplication on the ciphertext domain for similar plaintexts. FuzzyPoW is a two-party interactive protocol used by S to attest that C is indeed in possession of some specific files.

As illustrated in Fig. 1, to provide the secure encryption allowing ciphertext similarity measurement, a client Alice with a plaintext w' first applies the similarity-preserving hash algorithm with the input of two public parameters (i.e., r_1 and r_2) to generate a plaintext-dependent deduplication tag and a fuzzy key, respectively, in FuzzyMLE. According to the properties of similarity-preserving hash, similar plaintexts will map to the similar tags (e.g., τ_w and $\tau_{w'}$) while it is difficult to reversely recover the corresponding plaintext from a tag. Thus, such a tag can work as the file identity, allowing S to conduct similar or duplicate data queries in its database once receiving it from a client Alice. Since the tag is a short fingerprint of w' , it can reduce the communication overhead for data query between Alice and S . The fuzzy key, with a specific length (e.g., 128 bits), is used as the parameter for generating the seed of data encryption key. Observing that fuzzy keys from similar plaintexts are similar rather than identical, they cannot be directly applied as encryption keys to encrypt the related plaintext. Otherwise, S cannot conduct similarity verification on those ciphertexts with similar plaintexts. Therefore, a unique random key generation algorithm is designed by leveraging the fuzzy extractor. That is, Alice chooses some helping parameters and computes the random key for each plaintext. Here, the helping parameters are some random elements, which can help future users to generate the file decryption key. Later, another client Bob, with a plaintext w similar to w' (i.e., within a specific Hamming distance), can generate an identical key with the help parameters from S . To allow C_s and S to conveniently measure the similarity of ciphertexts, the encryption based on bit-wise XOR is employed in our solution. That is, FuzzyMLE adopts a pseudo-random generator which takes the random key seed as input to generate a random encryption key for the plaintext. Clearly, with the help of the fuzzy extractor, FuzzyMLE results in the identical encryption key for similar plaintexts from different clients and therefore leads to similar ciphertexts, enabling data similarity measurement.

To prevent the duplicate-faking attack, FuzzyPoW is proposed for S to securely verify whether a client C is in

possession of some similar ciphertext (i.e., within certain Hamming distance) before data deduplication. Since the encryption algorithm is malleable, this scheme needs to keep both parties' inputs private. In FuzzyPoW, a privacy-preserving OT protocol is leveraged to implement such functions. More precisely, for each bit (e.g., $c[i]$) in a ciphertext c , $\mathcal{C}$ first selects a random number r_i to blind $c[i]$ and obtains an element pair $(r_i + c[i], r_i + \bar{c}[i])$, which is sent to $\mathcal{S}$ through OT protocol. For each received element pair, $\mathcal{S}$ reveals only one element according to the i th bit of its ciphertext c' and adds all the revealed elements together to obtain $\sum_{i=1}^N r_i + \sum_{i=1}^N (c[i] \oplus c'[i])$. Here, N is the bit-number of ciphertexts. After that, $\mathcal{C}$ sends $\sum_{i=1}^N r_i$ to $\mathcal{S}$. The server subtracts $\sum_{i=1}^N r_i$ from $\sum_{i=1}^N r_i + \sum_{i=1}^N (c[i] \oplus c'[i])$ to obtain $\sum_{i=1}^N (c[i] \oplus c'[i])$ (i.e., the Hamming distance $\text{dis}_{\text{Ham}}(c, c')$) between c and c' . It is obvious that, if the leveraged OT protocol is secure, $\mathcal{S}$ only obtains $\text{dis}_{\text{Ham}}(c, c')$ and $\mathcal{C}$ obtains nothing. Thus, FuzzyPoW does not reveal any information about one party's input to the other party.

3.3 Preliminary Knowledge

3.3.1 Hash Functions & Oblivious Transfer

Cryptographic Hash Functions. Cryptographic hash functions h s are also called One-Way Hash Functions (OWHFs). h maps a bit string of an arbitrary length or a predetermined maximal length to a string of a fixed length. Given a $h : X \rightarrow Y$, we have the following three properties: (1) Given h and an input x , $h(x)$ is easy to compute; (2) Given $y \in Y$, it is computationally infeasible to find a value $x \in X$ such that $h(x) = y$; (3) Given $x \in X$, it is computationally infeasible to find a value $x' \in X$, such that $x' \neq x$ and $h(x') = h(x)$.

Similarity-Preserving Hash. Similarity-preserving hash $\tilde{h}$ is a specific type of hash algorithm, used to identify similar but not necessarily identical files. It takes an arbitrary-length bit string as input and output a fixed-length digest. It has two important properties. (1) Digest similarity: It will output similar digests if and only if the input data are similar (e.g., within certain Hamming distance). (2) One-wayness: If the digest of data is a hash of its features, it is difficult to recover the preimage of the digest. Since similarity-preserving hash is a key component of data secrecy protection, we assume the existence of the second property. A practical candidate is that, one can first extract all features of data along with their weights (e.g., the time they appear). Then, a l -counter vector $(v_1, \dots, v_l)$ is initialized, where l is the similarity-preserving hash fingerprint size (e.g., 64 bits or 128 bits). Each feature of the data is then hashed with a cryptographic hashing algorithm (e.g., SHA). If the bit at position i ($i \in \{1, \dots, l\}$) in the resulting cryptographic hash digest is 0, v_i is decremented by the weight of that feature; otherwise, v_i is incremented by the same weight. When all data features are processed, the similarity-preserving hash fingerprint is computed as follows: for all $i \in \{1, \dots, l\}$, if $v_i > 0$, set v_i to 1, and 0 otherwise.

Oblivious Transfer. Oblivious Transfer (OT) is a two-party protocol, which is first introduced by Rabin [30]. In an OT protocol, there are a *sender* and a *receiver*. The sender has a secret message and then sends it to the receiver. Then, the receiver receives the message with probability 1/2, while the sender does not know whether the message has been

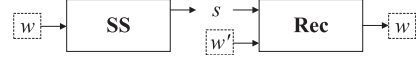


Fig. 2. Syntax depiction of secure sketch.

received or not. A slightly different primitive, called 1-out-of-2 Oblivious Transfer (OT_1^2) developed later in [31], is used in our scheme. A OT_1^2 enables a receiver to obtain 1-out-of-2 elements held by a sender without learning information on the other element and without the sender knowing which element has been chosen. OT extensions enable protocols to compute a very large number of OTs using a relatively small number of “base-OTs” [32]. To meet different security demands, the oblivious transfers of [33] and [34] are designed to protect against the semi-honest adversary and malicious adversary, respectively.

3.3.2 Hamming Distance & Error-Correcting Code

Hamming Distance. A metric space is a set $\mathcal{M}$ with a distance function $\text{dis} : \mathcal{M} \times \mathcal{M} \rightarrow [0, \infty)$. In our work, $\mathcal{M}$ will always be a finite set, and dis takes on only integer values. A hamming metric is $\mathcal{M} = \mathcal{F}^n$ for some alphabet $\mathcal{F}$, and we have the following definition of Hamming distance.

Definition 1 (Hamming Distance). For two string x and y over a finite alphabet $\mathcal{F}^n$, the Hamming distance, denoted $\text{dis}_{\text{Ham}}(x, y)$, is defined as the number of positions at which x and y differ, i.e., $\text{dis}_{\text{Ham}}(x, y) = |\{i | x[i] \neq y[i]\}|$.

We have $\text{dis}_{\text{Ham}}(x, y) = 0$ if and only if $x = y$, and $\text{dis}_{\text{Ham}}(\cdot, \cdot)$ obeys symmetry and the triangle inequality.

Error-Correcting Code. A code $\mathbf{C}$ is a subset $\{c_0, \dots, c_{n-1}\}$ of n elements of $\mathcal{M}$. The *encoding* is denoted by $\mathbf{C}(\cdot)$ which maps from $x_i \in \mathcal{M}$ to $c_i \in \mathbf{C}$. The *minimum distance* of $\mathbf{C}$ is the smallest $d > 0$ such that for all $i \neq j$ we have $\text{dis}(c_i, c_j) \geq d$. The *error-correcting distance* of $\mathbf{C}$ is the largest number $t > 0$ such that for every $x \in \mathcal{M}$ there exists at most one codeword c in the ball of radius t around x , e.g., $\text{dis}(x, c) \leq t$. For example, let $\mathcal{F}$ be a finite alphabet of size q . A subset $\mathbf{C} \subset \mathcal{F}^n$ is an $[n, k, d]_{\mathcal{F}}$ error-correcting code if it contains q^k elements such that $\text{dis}_{\text{Ham}}(c, c') \geq d$ for any $c, c' \in \mathbf{C}$. Given $x \in \mathcal{F}^n$, a *decoding* algorithm can find a unique element $c \in \mathbf{C}$ such that $\text{dis}_{\text{Ham}}(c, x) \leq (d-1)/2$. If $\mathcal{F}$ is a field and $\mathbf{C}$ is a linear subspace of the vector space $\mathcal{F}^n$, then $\mathbf{C}$ is a linear code. The syndrome syn over $\mathcal{F}$ is an $(n-k) \times n$ matrix whose kernel is $\mathbf{C}$. We define $\text{syn}(x)$ as a syndrome of a vector $x \in \mathcal{F}^n$. If the number of nonzero symbols in (i.e., Hamming weight of) x is at most $(d-1)/2$, x is unique given $\text{syn}(x)$, and it can be recovered by the unique decoding algorithm.

3.3.3 Secure Sketch & Fuzzy Extractor

Secure Sketch. Let $\mathcal{M}$ be a metric space with distance function dis . Fig. 2 describes the syntax of the secure sketch. It can be defined as follow:

Definition 2 (Secure Sketch [35]). An $(\mathcal{M}, m, \tilde{m}, t)$ -secure sketch is a pair of efficient randomized procedures (SS , Rec) (“sketch” and “recover”) such that:

- The sketching procedure SS takes $w \in \mathcal{M}$ as input and returns a string $s \in \{0, 1\}^*$.

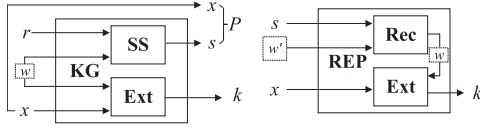


Fig. 3. Syntax depiction of fuzzy extractor.

- The recovery procedure Rec takes an element $w' \in \mathcal{M}$ and $s \in \{0, 1\}^*$.

It is correct, if $\text{dis}(w, w') \leq t$ then $\text{Rec}(w', \text{SS}(w)) = w$, and if $\text{dis}(w, w') > t$ then no guarantee is provided about the output of Rec . We say it is secure if for any distribution W over $\mathcal{M}$ with min-entropy m , the value of W can be recovered by the adversary who observes s with probability no greater than 2^{-m} .

Secure Sketch Based on Hamming Distance. To obtain a secure sketch for correcting Hamming errors over $\mathcal{F}^n$, we start with a $[n, k, 2t + 1]_{\mathcal{F}}$ error-correcting code C . The idea is to use C to correct errors in w even though w may not be in C , by shifting the code so that a codeword matches up with w , and storing the shift as the sketch. For example, we view $\mathcal{F}$ as an additive group. On input w , select a uniformly random chosen codeword $c \in C$, and set $s = \text{SS}(w) = w - c$ to be the shift needed to get from c to w . To compute $\text{Rec}(w', s)$, subtract s from w' to get $c' = w' - s$ and then decode c' to get c . Because $\text{dis}_{\text{Ham}}(w, w') \leq t$, so is $\text{dis}_{\text{Ham}}(c, c')$. Finally, we can obtain w by computing $w = c + s$.

Fuzzy Extractor. Fuzzy extractors enable the generation of a nearly uniform string k from w and its subsequent reproduction given any w' similar to w . The reproduction is computed with the help of the helper string P produced during the initial extraction. Since k is nearly uniform even given P , P needs not to remain secret. Fig. 3 describes the syntax of the fuzzy extractor. It can be defined as follow:

Definition 3 (Fuzzy Extractor [35]). An $(\mathcal{M}, m, \ell, t, \epsilon)$ -fuzzy extractor is a pair of efficient randomized procedures (KG, REP) (“key generate” and “reproduce”) such that:

- $\{k, P\} \leftarrow \text{KG}(w; r, x)$ is a user-side key generation algorithm. Given $w \in \mathcal{M}$ and two randomness r and x , it outputs an extracted string $k \leftarrow \text{Ext}(w; x)$ ($k \in \{0, 1\}^\ell$) and the helper string $P = (\text{SS}(w; r), x)$.
- $\{k\} \leftarrow \text{REP}(w', P)$ is a reproduction algorithm. It takes an element $w' \in \mathcal{M}$ and the string s as input and recovers $w = \text{Rec}(w', s)$. Finally, it outputs a nearly random key $k \leftarrow \text{Ext}(w; x)$.

According to [35], we have: (1) Assume that $\text{dis}(w, w') \leq t$ and $(k, P) \leftarrow \text{KG}(w; r, x)$, a fuzzy extractor is correct if $\text{Rep}(w', P) = k$; (2) Its secure property guarantees that for all m -sources W over $\mathcal{M}$, k is nearly uniform even given P (i.e., if $\tilde{H}_\infty(W|E) \geq m$), then $(k, P, E) \approx_\epsilon (U_\ell, P, E)$. Here, $\mathcal{M}$'s min-entropy is m ; (3) It is efficient if KG and REP finish in expected polynomial time.

3.4 Definitions of Our Building Blocks

Before we present the constructions of two key building blocks in FuzzyDedup, we first present the definitions that will be used in each building block.

3.4.1 Fuzzy Message-Locked Encryption (FuzzyMLE)

FuzzyMLE is an encryption algorithm used by C_s to encrypt their files before outsourcing them to S , but preserving the functionality of fuzzy deduplication. We give its definitions and concrete construction as follows.

Definition 4. FuzzyMLE is a collection of seven polynomial-time algorithms ($\text{PG}, \text{FKG}, \text{RKG}, \text{REP}, \text{ENC}, \text{DEC}, \text{TAG}$), defined as follows:

$\text{PG}(1^\lambda) \rightarrow \{r_1, r_2\}$ is a parameter generation algorithm run by the server, which takes a security parameter λ as input and outputs two random values r_1 and r_2 .

$\text{FKG}(1^\lambda, r_2, w) \rightarrow \tilde{k}_w$ is a fuzzy key generation algorithm executed by a client to compute the digest of input data. A security parameter λ , a random number $r_2 \in \{0, 1\}^\lambda$ and a file w are taken as the input and a fuzzy key $\tilde{k}_w$ is returned.

$\text{RKG}(1^\lambda, \tilde{k}_w; r_w) \rightarrow \{k_w, P_w\}$ is a randomized key generation algorithm run by the client. It takes a security parameter λ and a fuzzy key with random number $\tilde{k}_w; r_w$ as inputs and outputs a random key k_w and a set of help parameters $P_w = \{x_w, s_w\}$. r_w is used to emphasize the randomness, and we also add it to the input of RKG .

$\text{REP}(\tilde{k}_{w'}, P_w) \rightarrow k_{w'}$ is a key reproduction algorithm executed by the client to compute the random key for the input data. It takes the helper parameter P_w and a fuzzy key $\tilde{k}_{w'}$ as inputs, and returns the secret key $k_{w'} = k_w$ iff $\tilde{k}_{w'}$ is similar to $\tilde{k}_w$. Otherwise, it will return a random value.

$\text{ENC}(k_w, w) \rightarrow c_w$ is an encryption algorithm executed by the client for data encryption. It takes a file w and a secret key k_w as inputs, and returns the ciphertext c_w .

$\text{DEC}(k_{w'}, c_w) \rightarrow w$ is an algorithm executed by the client for data decryption. It takes a secret key $k_{w'}$ and the ciphertext c_w as inputs, and returns the plaintext w .

$\text{TAG}(1^\lambda, r_1, w) \rightarrow \tau_w$ is a tag generation algorithm executed by the client to generate the tag for input data. It takes a security parameter λ , a public random number r_1 and a file w as inputs, to return a file tag τ_w .

FuzzyMLE satisfies the correctness and security properties:

Correctness: Let $\tilde{k}_w$ and $\tilde{k}_{w'}$ be the fuzzy keys of w and w' , respectively. Suppose (k_w, P_w) is generated by $\text{RKG}(1^\lambda, r_w, \tilde{k}_w)$ and $k_{w'}$ is generated by $\text{REP}(\tilde{k}_{w'}, P_w)$. The correctness property of FuzzyMLE guarantees that if $\text{dis}_{\text{Ham}}(\tilde{k}_w, \tilde{k}_{w'}) \leq t$ then $w = \text{DEC}(k_{w'}, \text{ENC}(k_w, w))$, else $k_{w'}$ is statistically close to a random r_w over $\{0, 1\}^{|k_w|}$, i.e., the statistical distance $\text{SD}((k_{w'}, P_w), (r_w, P_w)) \leq \epsilon$.

Privacy: Similar to PRV\$-CDA in [18], we formalize the notion of FMLE privacy Fuzzy-PRV\$-CDA for unpredictable messages, where “CDA” represents “Chosen-Distribution Attack”. Let PG be the parameter generation algorithm. We also assume a source (a polynomial-time algorithm) $\mathcal{M}$ is unpredictable, where messages are unpredictable given the auxiliary information. The algorithm $\mathcal{M}$ inputs 1^λ and returns $(\mathbf{W}, Z)$ where $\mathbf{W}$ is a vector over $\{0, 1\}^*$ and $Z \in \{0, 1\}^*$. We require that the vector has the length of $m(\lambda)$ for some function m called as the number of messages of the source. We require that there is a function len , called the message length of the source, such that the string $\mathbf{W}[i]$ has the length of $\text{len}(\lambda, i)$ for all $i \in [m(\lambda)]$. We require that $\mathbf{W}[i_1] \neq \mathbf{W}[i_2]$ for all distinctive $i_1, i_2 \in [m(\lambda)]$, meaning the entries of each vector are distinct. We refer to Z as the auxiliary information. In the games of Fig. 4, if $\mathcal{A}$ is an adversary, we let $\text{Adv}_{\text{FMLE}, \mathcal{M}, \mathcal{A}}^{\text{fuzzy-priv-cda}}(\lambda) = 2 \cdot$

```

main Fuzzy-PRV$-CDAFMLE, MA(λ)
{r1, r2} ← PG(1λ); b ← {0, 1}; (W, Z) ← M(1λ)
for i = 1, ..., |W| do
  kW[i] ← FKG(r2, W[i])
  {kW[i], PW[i]} ← RKG(1λ, kW[i]; rW[i])
  C1[i] ← ENC(kW[i], W[i]) = G(kW[i]) ⊕ W[i]
  C0[i] ← {0, 1}|C1[i]|
b' ← A(r1, r2, PW, C0, Z)
return b = b'

```

Fig. 4. Game defining of Fuzzy-PRV\$-CDA for FMLE.

$\Pr[\text{Fuzzy-PRV\$-CDA}_{\text{FMLE}, \mathcal{M}}^A(\lambda)] - 1$. We say that FMLE is Fuzzy-PRV\$-CDA secure over $\mathcal{M}$ if $\text{Adv}_{\text{FMLE}, \mathcal{M}, A}^{\text{fuzzy-prv\$-cda}}(\cdot)$ is negligible.

3.4.2 Fuzzy Proof of Ownership Scheme (FuzzyPoW)

Similar to the PoW schemes [36] which allow the server S to verify a client C in possession of an identical file, FuzzyPoW is designed for S to attest that C owns a similar one. However, the design of secure FuzzyPoW based on FuzzyMLE is nontrivial, where the techniques (e.g., Hash and Merkle Tree [37]) used for secure data equality verification is infeasible to solve the secure data similarity measurement. Moreover, FuzzyMLE is based on malleable encryption (i.e., bit-wise XOR encryption), which is not secure when C directly sends his/her ciphertext to S for similar duplicate ownership proof. More precisely, assume that $c_w = w \oplus G(k_w)$ and $c_{w'} = w' \oplus G(k_{w'})$ are ciphertexts of FuzzyMLE held by S and C , respectively. Once obtaining both c_w and $c_{w'}$, S will be able to compute $w \oplus w' = c_w \oplus c_{w'}$, which will leak the XOR bits of w and w' to the S . To solve the above challenges, a new fuzzy-style PoW scheme has to be designed for secure data ownership proof among similar ciphertexts. Before giving the concrete definition and construction of FuzzyPoW, we provide functionality definitions of Hamming distance measurability and similarity security.

Definition 5 (Hamming Distance Measurability). Fuzzy-PoW is Hamming distance measurable if a client and the server take a plaintext w' and c_w (i.e., the ciphertext of w) as input, respectively, and the client can efficiently prove the Hamming distance between w and w' to the server.

Definition 6 (Similarity Security). Suppose a client has a plaintext w' and the server has a record of ciphertext c_w . Fuzzy-PoW is similarity secure if each party cannot obtain any information about the other's input while only the server can measure the distance between $c_{w'}$ (i.e., the ciphertext of w') and c_w .

Formally, FuzzyPoW is a two-party interactive protocol, where a client C wants to prove to the server S that he/she is in possession of a similar file. It can be defined as follows.

$$\text{FuzzyPoW} : [C : w'; S : c_w] \rightarrow [C : \perp; S : \text{dis}_{\text{Ham}}(c_w, c_{w'})].$$

Here, C is the owner of w' and S stores c_w from other users. C first sends w' 's tag $\tau_{w'}$ to S and proves to S that $\tau_{w'}$ and τ_w are similar. Then, S asks C to launch the FuzzyPoW. Finally, S only obtains $\text{dis}_{\text{Ham}}(c_w, c_{w'})$ while C obtains nothing. S believes that C owns a similar file if $\text{dis}_{\text{Ham}}(c_w, c_{w'}) < T$, where T is a threshold value that can be specified according to various practical applications.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on April 19, 2026 at 15:56:22 UTC from IEEE Xplore. Restrictions apply.

The security of FuzzyPoW can be considered as the security of two-party protocols, which informally can be ensured by simulating the protocol in an ideal model and showing that all adversarial behaviors in a real execution are simulatable in this model, namely indistinguishability between a real execution and an ideal execution of the protocol. In the ideal model, the inputs of both parties are sent to a trusted third party who controls the computation and sends the outputs back to the respective parties. In FuzzyPoW, since only one party (i.e., S) receives the output while the other party (i.e., C) learns nothing, we can achieve a weaker notion of security against malicious S . That is, the protocol achieves one-sided simulatability security if it is private against a semi-honest C and fully-secure against malicious S , as defined in [38].

We denote FuzzyPoW π between C and S for computing Hamming distance $f : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ designated only to S . Consider a malicious and admissible adversary A who corrupts C or S . Then, we can denote the FuzzyPoW (i.e., π) as $\pi(x, y) = (\emptyset, f_S(x, y))$ with only a single output $f_S(x, y)$ for S . Let $\text{REAL}_{\pi, A(z), \mathcal{P}}(x, y, \lambda)$ denote the outputs of the honest party and the adversary A (controlling party $\mathcal{P}$) after a real execution of protocol π , where C has the input x , S has the input y , A has the auxiliary input z , and the security parameter is λ . Let $\text{IDEAL}_{f, B(z), \mathcal{P}}(x, y, \lambda)$ be the analogous distribution in an ideal execution with a trusted party that computes f for the parties and sends the output to S only. Finally, let $\text{VIEW}_{\pi, A(z), \mathcal{P}}^A(x, y, \lambda)$ be the view of the adversary after a real execution of π as above, then we have the definition of one-sided simulatability security as follow:

Definition 7 (One-sided Simulatability Security). Let f be Hamming distance computation algorithm where only S receives the output. We say that FuzzyPoW π securely computes f with one-sided simulation against non-uniform PPT adversary A if the following conditions hold:

1. For every A controlling S in the real model, there exists a non-uniform PPT adversary B for the ideal model, yielding

$$\{\text{REAL}_{\pi, A(z), S}(x, y, \lambda)\}_{x, y, z, \lambda} \stackrel{c}{=} \{\text{IDEAL}_{f, B(z), S}(x, y, \lambda)\}_{x, y, z, \lambda}. \quad (1)$$

2. For every A controlling C , we have

$$\{\text{VIEW}_{\pi, A(z), C}^A(x, y, \lambda)\}_{x, y, y', z, \lambda} \stackrel{c}{=} \{\text{VIEW}_{\pi, A(z), C}^A(x, y', \lambda)\}_{x, y, y', z, \lambda}. \quad (2)$$

Here, $\lambda \in \mathbb{N}$, $x, y, y', z \in \{0, 1\}^*$ and $|x| = |y| = |y'|$. y and y' indicate the inputs of S .

Since FuzzyPoW relies on OT protocol, OT hybrid model [39] can be used to prove the security of our proposed protocol. This model is a hybrid of the real and ideal models, where the parties interact with each other and have access to a trusted party (ideal functionality) that executes the OT protocol for them. It suffices to analyze the security of our protocol in a hybrid model, where we only need to prove that the executions between hybrid model and ideal model are indistinguishable.

3.5 Our Constructions

After presenting the necessary definitions in Section 3.4, we next illustrate the constructions of FuzzyMLE and FuzzyPoW, before we elaborate the FuzzyDedup construction.

3.5.1 FuzzyMLE Construction

Following the procedure in Fig. 1, consider that w and w' are plaintexts from two clients (i.e., Bob and Alice). $r_1 \in \{0, 1\}^\lambda$ and $r_2 \in \{0, 1\}^\lambda$ are random numbers selected by $\mathcal{S}$ using PG algorithm and published to all clients.

For the first data owner, e.g., Alice, she runs FKG and TAG algorithms, which employ similarity-preserving hash algorithm $\tilde{h}$ to compute the fuzzy key $\tilde{k}_{w'} \leftarrow \tilde{h}(r_2, w')$ and tag $\tau_{w'} \leftarrow \tilde{h}(r_1, w')$ for w' , respectively. Since $\tilde{h}$ is not a keyed algorithm, its input is padded with the random number, i.e., $\tilde{k}_{w'} \leftarrow \tilde{h}(w' \oplus G(r_2))$ and $\tau_{w'} \leftarrow \tilde{h}(w' \oplus G(r_1))$, respectively. Here, $G(\cdot)$ is a pseudo-random number generator that generates a $|w'|$ -bit random number. Alice uses KG of the fuzzy extractor (in Section 3.3.3) to generate the random key $r_{w'}$ and computes $\{P_{w'}, k_{w'}\} \leftarrow \text{RKG}(1^\lambda, \tau_{w'}, \tilde{k}_{w'})$. During this process, KG calls the secure scratch algorithm $s_{w'} \leftarrow \text{SS}\{r_{w'}, w\}$ and extraction algorithm $k_{w'} \leftarrow \text{Ext}\{w', x_{w'}\}$ to generate the help parameters $P_{w'} = \{x_{w'}, s_{w'}\}$ and a random encryption key $k_{w'}$, respectively. Here, $x_{w'}$ is a parameter published later and $r_{w'}$ is a random parameter privately selected by Alice. $r_{w'}$ is used to generate a random code $c \leftarrow C(r_{w'})$ for the secure scratch algorithm, where $C(\cdot)$ is a code generation algorithm. Then, by using bit-wise XOR encryption, Alice can encrypt the plaintext w' , i.e., $c_{w'} = w' \oplus G(k_{w'}) = \text{ENC}(k_{w'}, w')$, and decrypt the ciphertext $c_{w'}$ with $k_{w'}$ as the symmetric key, i.e., $w' = c_{w'} \oplus G(k_{w'}) = \text{DEC}(k_{w'}, c_{w'})$. Finally, Alice stores $\tau_{w'}$, $P_{w'}$ and $c_{w'}$ to $\mathcal{S}$'s database, and $\mathcal{S}$ registers Alice to $c_{w'}$ and sends her a link $Link_{k_{w'}}$ for data access.

For the later data owner, e.g., Bob, he first computes its tag $\tau_w \leftarrow \tilde{h}(r_1, w)$ and fuzzy key $\tilde{k}_w \leftarrow \tilde{h}(r_2, w)$ for plaintext w . τ_w is then sent to $\mathcal{S}$ for database query. Assume that $\mathcal{S}$ identifies the record with tag $\tau_{w'}$ from Alice. Then, Bob runs the key reproduction algorithm REP with the help parameter $P_{w'}$ from $\mathcal{S}$. Similar to the fuzzy extractor, REP takes $P_{w'}$ and $\tilde{k}_w$ as input, and returns a symmetric key $k_{w'}^* \leftarrow \text{REP}(\tilde{k}_w, P_{w'})$ and $k_{w'}^* = k_{w'}$ iff $\tilde{k}_w$ is similar to $\tilde{k}_{w'}$. Bob keeps $k_{w'}^*$ secrecy at local storage and encrypts w to obtain ciphertext $c_w = \text{ENC}(k_{w'}^*, w)$. Then, Bob and $\mathcal{S}$ launch the data ownership proof protocol (elaborated in next section) with $c_{w'}$ and c_w as input, respectively. If Bob passes the test, $\mathcal{S}$ registers Bob to $c_{w'}$ and sends him the $Link_{k_{w'}}$ for data access. It is obvious that, if w and w' are similar, Bob can obtain the decryption key $k_{w'}$ of $c_{w'}$ and pass the data ownership proof. Otherwise, he can only obtain a random key $k_{w'}^*$, which cannot decrypt any ciphertext.

In MLE [18], tag consistency is important when a client Alice is malicious, which does not upload a dishonest encryption of w' , but a maliciously-generated ciphertext $c_{w'}^*$. In such a scenario, when Bob with w tries to upload c_w , the server sees a tag match $\text{dis}_{\text{Ham}}(\tau_{w'}, \tau_w)$ and thus keeps only $c_{w'}$ while deleting $c_{w'}$. Since MLE is a deterministic encryption scheme, it is possible to support tag consistency by computing the tag directly from the corresponding ciphertext. However, FuzzyMLE is a randomized encryption

scheme, which cannot be expected to provide the tag consistency like MLE. Instead, FuzzyDedup provides a fuzzy style proof of ownership scheme, which allows the subsequent data owner to check whether the server-side ciphertext outsourced by a previous user is consistent with its tag. We will be elaborated this in the following section.

3.5.2 FuzzyPoW Construction

Assume that $\mathcal{C}$ is the owner of w' , and $\mathcal{S}$ holds the ciphertext $c_w = (c_w[1], \dots, c_w[|w'|])$ stored by some other cloud users. $\mathcal{S}$ asks $\mathcal{C}$ to prove the *exact* Hamming distance between ciphertext $c_w = (c_w[1], \dots, c_w[|w'|])$ and $c_{w'}$ through FuzzyPoW, where $c_{w'}$ is the ciphertext of w' encrypted using FuzzyMLE. Here, $\mathcal{C}$ wants to protect $c_{w'}$, not revealing any bit of $c_{w'}$ to $\mathcal{S}$.

The construction details of FuzzyPoW is depicted in Algorithm 1, where the secure two-party Hamming distance computation protocol proposed in [40] is leveraged to securely compute the Hamming distance $\text{dis}_{\text{Ham}}(c_w, c_{w'})$, i.e., the number of bits where $c_{w'}[i] \oplus c_w[i] = 1$, for $\mathcal{S}$.

Algorithm 1. Fuzzy Proof of Ownership (FuzzyPoW)

Input: $\mathcal{C}$ inputs a $|c_{w'}|$ -bit string $c_{w'} = (c_{w'}[1], \dots, c_{w'}[|w'|])$;

$\mathcal{S}$ inputs a $|c_w|$ -bit string $c_w = (c_w[1], \dots, c_w[|w'|])$.

We assume that $|w'| = |w| = N$.

Output: $\mathcal{S}$ outputs $\text{dis}_{\text{Ham}}(c_{w'}, c_w)$ and $\mathcal{C}$ outputs $\perp$.

Protocol:

1. $\mathcal{C}$ generates N random values $\{r_1, \dots, r_N\} \in_R \mathbb{Z}_{N+1}$ and computes $\Sigma_r = \sum_{i=1}^N r_i$.
 2. For each $i = 1, \dots, N$, $\mathcal{C}$ and $\mathcal{S}$ engage in a OT_1^2 where $\mathcal{C}$ acts as the sender and $\mathcal{S}$ as the receiver.
 - a. The input of $\mathcal{C}$ is $(r_i + c_{w'}[i]; r_i + \overline{c_{w'}[i]})$.
 - b. The selection bit of $\mathcal{S}$ is $c_w[i]$.
 - c. The output obtained by $\mathcal{S}$ is consequently a result $t_i = r_i + (c_{w'}[i] \oplus c_w[i])$.
 3. $\mathcal{S}$ computes $\Sigma_t = \sum_{i=1}^N t_i$.
 4. $\mathcal{C}$ sends Σ_r to $\mathcal{S}$.
 5. $\mathcal{S}$ computes $\text{dis}_{\text{Ham}}(c_{w'}, c_w)$ by outputting $\Sigma_t - \Sigma_r$.
-

In FuzzyPoW, $\mathcal{C}$ works as the sender, which first generates N random values $\{r_1, \dots, r_N\} \in_R \mathbb{Z}_{N+1}$ as the blinding factors and launches N OT interactions with $\mathcal{S}$. For the i th transfer, $\mathcal{C}$ sends the message $(r_i + c_{w'}[i], r_i + \overline{c_{w'}[i]})$ to $\mathcal{S}$. $\mathcal{S}$ acts as the receiver for all these N OT interactions. Upon receiving $(r_i + c_{w'}[i], r_i + \overline{c_{w'}[i]})$, $\mathcal{S}$ selects $r_i + c_{w'}[i]$ if $c_w[i] = 0$; $r_i + \overline{c_{w'}[i]}$, otherwise. That is, for all the N OT's, $\mathcal{S}$ obtains $r_i + (c_w[i] \oplus c_{w'}[i]) \bmod (N+1)$ ($1 \leq i \leq N$). Then, $\mathcal{S}$ adds all these values to get $\Sigma_t = \sum_{i=1}^N r_i + \sum_{i=1}^N (r_i \oplus y_i) = \Sigma_r + \text{dis}_{\text{Ham}}(c_{w'}, c_w)$ where $\Sigma_r = \sum_{i=1}^N r_i$. Finally, $\mathcal{C}$ computes Σ_r and sends it to $\mathcal{S}$. The final output of $\mathcal{S}$ will be $\text{dis}_{\text{Ham}}(c_{w'}, c_w) = \Sigma_t - \Sigma_r$. If the designed Hamming distance threshold is T and $\text{dis}_{\text{Ham}}(c_{w'}, c_w) < T$, $\mathcal{S}$ considers that $c_{w'}$ and c_w are similar and hence $\mathcal{C}$ is in possession of a data that is similar to the plaintext of c_w . Here, we allow data owners to have different T s, which enable our deduplication much flexible while can balance the data quality and deduplication ratio according to his/her willingness. Otherwise, $\mathcal{S}$

considers that $\mathcal{C}$ does not have a plaintext whose ciphertext is similar to c_w .

The above FuzzyPoW is secure when $\mathcal{C}$ is semi-honest. However, from Algorithm 1, we can see a malicious $\mathcal{C}$ may cheat $\mathcal{S}$ to pass the data ownership proof through sending the forged Σ_r . To deal with such an attack, FuzzyPoW should provide full security in the malicious setting. Fortunately, the committed OT [40] can be leveraged to achieve this purpose. By adopting the committed OT, FuzzyPoW guarantees that the inputs of $\mathcal{C}$ are consistent and the outputs of $\mathcal{S}$ based on $\mathcal{C}$'s inputs follow the protocol after the OTs (i.e., without being changed by any malicious party). In terms of efficiency, FuzzyPoW is cost-inefficient, where the computation and communication overheads are linear with the bit length of the ciphertext. To further improve its efficiency, we design two variants for FuzzyPoW, namely probabilistic FuzzyPoW and hashing FuzzyPoW, which will be elaborated in Section 4.

3.5.3 FuzzyDedup Construction

Assume that clients $\mathcal{C}$ s are the data owners and they want to outsource their data to a remote cloud storage server $\mathcal{S}$ in a secure way while allowing to conduct client-side similar data deduplication. For $\mathcal{C}$ s, they only need to keep the identity links and the encryption keys of the data at a local database for further data downloading from $\mathcal{S}$ and decryption. For $\mathcal{S}$, a database $\mathbb{DB}$ is maintained to record all the information related to the data from $\mathcal{C}$ s. In our system, the cipher pool table of $\mathbb{DB}$ is used to store ciphertexts from $\mathcal{C}$ s. The FuzzyDedup table of $\mathbb{DB}$ provides some necessary fields, such as Hamming weight, tag, and link, to store relative information for each ciphertext from $\mathcal{C}$ s so as to identify it when needed.

FuzzyDedup includes four core protocols, i.e., Setup, Upload, Download and Delete, where the latter three are interactive protocols. They are constructed as follows.

Setup : This protocol is run by a client $\mathcal{C}$. Assume Alice is the first client that uploads the ciphertext of data w' to the cloud server $\mathcal{S}$. She first runs the tag generation algorithm $\tau_{w'} \leftarrow \text{TAG}(1^\lambda, r_1, w')$ and the fuzzy key generation algorithm $\tilde{k}_{w'} \leftarrow \text{FKG}(1^\lambda, r_2, w')$ to generate file tag and fuzzy key of w' , respectively, i.e., $\tau_{w'} \leftarrow \tilde{h}(r_1 \oplus w')$ and $\tilde{k}_{w'} \leftarrow \tilde{h}(r_2 \oplus w')$. Alice can then select a random code $c \in \mathcal{C}$, and call the random key generation algorithm $\{k_{w'}, P_{w'}\} \leftarrow \text{RKG}(1^\lambda, r_{w'}, \tilde{k}_{w'})$ to generate a random key $k_{w'}$ and the helper parameter set $P_{w'}$.

Upload : This protocol is run between $\mathcal{C}$ and $\mathcal{S}$. Initially, $\mathcal{C}$ computes the tag $\tau_{w'}$ and sends it to $\mathcal{S}$, where it will be used to conduct fuzzy duplication query in $\mathbb{DB}$. Based on whether $\tau_{w'}$ exists in $\mathcal{S}$'s database or not, $\mathcal{S}$ launches the following two different protocols with $\mathcal{C}$:

▷ *Duplication not found:* If $\mathcal{S}$ does not have a tag that is similar to $\tau_{w'}$ in $\mathbb{DB}$, then Upload protocol runs as follows:

$$\text{Upload} : [\mathcal{C} : \tau_{w'}, w', r_{w'}; \mathcal{S} : \mathbb{DB}] \rightarrow [\mathcal{C} : k_{w'}, c_{w'}, P_{w'}, \text{Link}_{w'}; \mathcal{S} : \tau_{w'}, P_{w'}, c_{w'}, \text{Link}_{w'}].$$

where $\mathcal{C}$ calculates a random eraser code $c_{r_{w'}} \leftarrow \mathcal{C}(r_{w'})$ with a randomly selected value $r_{w'}$ and then runs the random key generation algorithm $\{k_{w'}, P_{w'}\} \leftarrow \text{RKG}(1^\lambda, r_{w'}, \tilde{k}_{w'})$ to

generate the random encryption key and the helper parameters. In addition, this client encrypts w' to get the ciphertext $c_{w'} = \text{ENC}(k_{w'}, w')$ and sends $\{\tau_{w'}, P_{w'}, c_{w'}\}$ to $\mathcal{S}$. $\mathcal{S}$ stores them in its database and returns a link $\text{Link}_{w'}$ to $\mathcal{C}$ for further downloading of $c_{w'}$. After that, $\mathcal{S}$ registers $\mathcal{C}$ to $c_{w'}$.

▷ *Duplication found:* If $\mathcal{S}$ has a record c_w whose tag τ_w is similar to $\tau_{w'}$ in $\mathbb{DB}$, the Upload protocol runs as follows:

$$\text{Upload} : [\mathcal{C} : \tau_{w'}, w'; \mathcal{S} : \mathbb{DB}, P_w] \rightarrow [\mathcal{C} : k_w, c_{w'}, \text{Link}_w; \mathcal{S} : \text{Link}_w].$$

Here, $\mathcal{S}$ returns the helper parameters $P_w = \{x_w, s_w\}$ to $\mathcal{C}$ according to τ_w . When $\mathcal{C}$ receives P_w , it first runs the key reproduction algorithm $k_w \leftarrow \text{REP}(\tilde{k}_{w'}, P_w)$ as described in Section 3.3.3. In this process, $\mathcal{C}$ first computes the code $c_{r_w} = \text{REC}(s_w, \tilde{k}_{w'})$ according to the error-correcting character of the code in Section 3.3.2, and then computes the random key by running the fuzzy extractor, i.e., $k_w \leftarrow \text{Ext}(x_w, \tilde{k}_{w'})$. After that, $\mathcal{C}$ encrypts w' by running the encryption algorithm $c_{w'} = \text{ENC}(k_w, w')$.

Later, $\mathcal{S}$ verifies that $\mathcal{C}$ is actually in possession of w' through FuzzyPoW if $c_{w'}$ is similar to c_w , according to its construction details presented in Section 3.4.2. Finally, $\mathcal{S}$ will send a link Link_w to $\mathcal{C}$ in a secure way for data access. If one or more similar tags are found in $\mathcal{S}$, FuzzyPoW will conduct over all tags until finding a matched one. Otherwise, $\mathcal{S}$ outputs “duplicate not found” and asks $\mathcal{C}$ to upload data. $\mathcal{S}$ registers $\mathcal{C}$ to the duplicate ciphertext as the upload one.

$$\text{Download} : [\mathcal{C} : \tau_w, \text{Link}_w, k_w; \mathcal{S} : \mathbb{DB}] \rightarrow [\mathcal{C} : w; \mathcal{S} : \perp].$$

Download : This protocol is an interactive protocol initialized by $\mathcal{C}$ to acquire data from $\mathcal{S}$. When $\mathcal{C}$ requests to download the ciphertext of w from $\mathcal{S}$, she/he first sends τ_w and Link_w to $\mathcal{S}$ through a secure channel. After receiving the request, $\mathcal{S}$ queries $\mathbb{DB}$ according to τ_w , locates c_w through Link_w and verifies that whether $\mathcal{C}$ is registered to c_w . If not, the request is ignored and $\mathcal{C}$ receives $\perp$. Otherwise, $\mathcal{S}$ will send c_w to $\mathcal{C}$. Once receiving c_w , $\mathcal{C}$ runs the decryption algorithm to get $w \leftarrow \text{DEC}(k_w, c_w)$ with the local private key k_w . In $\text{DEC}(k_w, c_w)$, $\mathcal{C}$ first runs the pseudo-random generator algorithm to generate the decryption key $G(k_w)$ and then calculates the plaintext $w = c_w \oplus G(k_w)$.

$$\text{Delete} : [\mathcal{C} : \tau_w; \mathcal{S} : \mathbb{DB}] \rightarrow [\mathcal{C} : \perp; \mathcal{S} : \mathbb{DB}'].$$

Delete : This protocol allows $\mathcal{C}$ to cancel his file registration or delete a file in $\mathcal{S}$. $\mathcal{C}$ sends a file tag τ_w to $\mathcal{S}$ for checking whether τ_w is in the tag field of $\mathbb{DB}$ and whether $\mathcal{C}$ is registered to τ_w . If not, the request is simply ignored and $\mathcal{C}$ receives $\perp$. Otherwise, $\mathcal{S}$ will update its $\mathbb{DB}$ to $\mathbb{DB}'$ through deleting the registration of $\mathcal{C}$ from τ_w . If no other client is registered to τ_w , $\mathcal{S}$ will delete the record with tag τ_w from $\mathbb{DB}$.

4 SECURITY AND EFFICIENCY IMPROVEMENTS

In this section, we propose various solutions to further enhance the security and efficiency of FuzzyDedup.

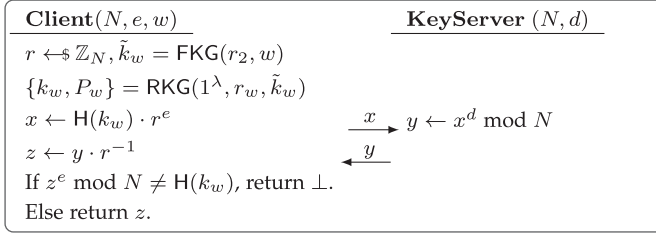


Fig. 5. Server-aided FuzzyMLE.

4.1 Security Enhancements of FuzzyMLE

It has been proved that no content-dependent encryption scheme can achieve semantic security [41], [42]. Similar to the existing MLE schemes, FuzzyMLE with deterministic tags is susceptible to off-line brute-force dictionary attacks [19]. For example, if the plaintext is from a dictionary set $= \{w_1, \dots, w_n\}$ of size n , the attacker can recover w from ciphertext c_w in the time of $n = |\text{set}|$. To avoid such an attack, we improve the security of FuzzyMLE by introducing the server-aided FuzzyMLE and random-tag FuzzyMLE.

Server-Aided FuzzyMLE. We integrate the existing server-aided deduplication scheme [19] based on RSA blind signatures [43], [44] into our constructed FuzzyMLE to protect against the off-line brute-force attack. A key server is introduced in our system and an RSA key generation algorithm is leveraged to produce N and d with the input of e , such that $ed \equiv 1 \bmod \phi(N)$. Modulus N is the product of two distinct primes with roughly equal length and $N < e$. (N, e) and (N, d) serve as the public key and private key pairs of the key server. The server-aided protocol based on RSA blind signature between the client and key server is shown in Fig. 5.

For a legal client, he will register to the key server. Assume the client's plaintext is w , he first randomly selects a number r and computes its random key k_w through running the fuzzy key generation algorithm FKG and random key generation algorithm RKG. Then, by inputting k_w and the key server's public key (N, e) , the client computes $x \leftarrow H(k_w)$ and blinds it with a random group element r^e . Here, $H: \{0, 1\}^* \rightarrow \mathbb{Z}_N$ is a cryptographic hash. The resulting blinded value, denoted as x , is sent to the key server, who will sign it by computing $y \leftarrow x^d \bmod N$, and send y back to the client. The client then removes the blinding element r by computing $z \leftarrow y \cdot r^{-1} \bmod N$, and verifying $z^e \bmod N = H(k_w)$. Finally, the output of z is further used to derive the secret encryption key of w and a data similarity verification tag $\tau_w = \tilde{h}(G(z))$ by using the pseudo-random key generation algorithm G and a similarity-preserving hash algorithm $\tilde{h}$, respectively.

Since the server-aided FuzzyMLE is oblivious, the key server cannot derive any information of the encryption key. Also, the protocol enables the client to verify the correctness of the computation performed by the key server. If an adversary does not have the privilege of controlling key server, the security of the encryption key is similar to the deterministic encryption schemes. Server-aided FuzzyMLE slows down the key-generation brute-force attack to on-line brute-force attack, where an attacker has to request for key generation from the key server. Similar to [19], the security of server-aided FuzzyMLE will reduce to that of the FuzzyMLE when facing a curious key server. Recently, based

on commitment schemes, Camenisch *et al.* [45], designed an ownership-based oblivious PRF against the malicious key server. However, it remains challenging to apply for building secure and efficient fuzzy similar data deduplication.

Random-Tag FuzzyMLE. For tag security, we can let the query tag of each data (e.g., w) to be derived from its random encryption key k_w . That is, τ_w is computed from $\tau_w = (g^r, g^{r \cdot h(k_w)})$, where g is a generator of a bilinear group, $h(\cdot)$ is a strong collision-resistant hash function (e.g., SHA-1 or SHA-256), and r is chosen uniformly at random.

Suppose the data from client $\mathcal{C}$ is w' . $\mathcal{C}$ first computes $\tilde{k}' \leftarrow \text{FKG}(r_2, w')$ and then runs the key reproduce algorithm $k_{w_i} \leftarrow \text{REP}(\tilde{k}', P_{w_i})$ over each record in $\mathcal{S}$'s database to produce the random key k_{w_i} . Finally, for each record, $\mathcal{C}$ verifies that whether there exists c_{w_i} in $\mathcal{S}$'s database with $\tau_{w'} = \text{TAG}(1^\lambda, k_{w_i})$ or not. In other words, given two tags $\tau_w = (g^r, g^{r \cdot h(k_w)})$ and $\tau_{w'} = (g^{r'}, g^{r' \cdot h(k_{w'})})$, $\mathcal{S}$ runs the equality-testing algorithm and verifies

$$\hat{e}(g^{r_w}, g^{r_{w'} \cdot h(k_{w'})}) \stackrel{?}{=} \hat{e}(g^{r_{w'}}, g^{r_w \cdot h(k_w)}).$$

It has been proved that the security of random-tag FuzzyMLE relies on the entropy-based DDH assumption [20], [46], where the tags do not reveal the additional information rather than that from its necessary functionality. We need to remark that the data similarity test is $O(n)$ in random-tag FuzzyMLE, where n is the number of records.

4.2 Efficiency Improvement of FuzzyPoW

Probabilistic FuzzyPoW (P-FuzzyPoW). FuzzyPoW requires the parties to deal with all bits of a file through OT protocol, which thus is not efficient. We can improve FuzzyPoW by relying on a probabilistic algorithm which selectively reveals the distance among files via random sampling.

Assume that $|c_w| = |c_{w'}| = N$ and λ is a security parameter. $\tilde{h}$ is a pseudo-random permutation (PRP) defined as follows:

$$\tilde{h}: \{0, 1\}^\lambda \times \{0, 1\}^{\log_2(N)} \rightarrow \{0, 1\}^{\log_2(N)}.$$

As illustrated in Algorithm 2, $\mathcal{S}$ selects the number of detection bits θ over the ciphertext, generates a random value $\gamma \leftarrow \{0, 1\}^\lambda$ and then sends it to $\mathcal{C}$. $\mathcal{C}$ and $\mathcal{S}$ can compute common positions. For example, with $c_{w'}$ and c_w , $\mathcal{C}$ and $\mathcal{S}$ run FuzzyPoW over the θ randomly selected common positions.

Algorithm 2. Random Selection of Bit Positions

Input: $\mathcal{C}$ and $\mathcal{S}$ take security parameter λ , string length N , number of sampled bits θ , and random number γ as inputs.

Output: θ random bit positions $\{p_1, \dots, p_\theta\}$.

for $i \leftarrow 1$ **to** θ **do**

Compute $p_i = \tilde{h}_\gamma(i)$;

Assume the server finally obtains a Hamming distance $\text{dis}_{\text{Ham}}'(c_{w'}, c_w)$ over the sampled θ positions. According to the law of large numbers, the observed mean from a large number of observations of a random variable will always approach the distribution mean. Thus, we can compute the Hamming distance of c_w and $c_{w'}$ via

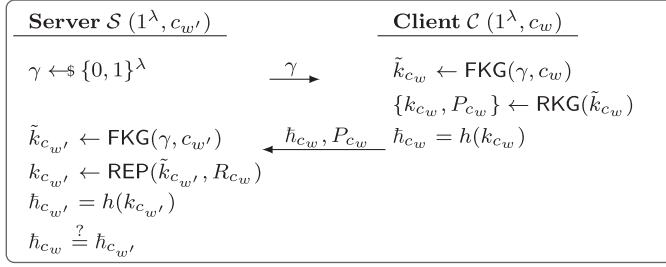


Fig. 6. Hashing FuzzyPoW.

$$\text{dis}_{\text{Ham}}(c_{w'}, c_w) \approx N \cdot \frac{\text{dis}_{\text{Ham}}'(c_{w'}, c_w)}{\theta}. \quad (3)$$

If $\text{dis}_{\text{Ham}}(c_{w'}, c_w) < T$, we consider that $c_{w'}$ and c_w are similar and hence C is in possession of a data that is similar to the plaintext of c_w . By sacrificing some accuracy, this solution is efficient by reducing the Hamming distance computation to those sampled bit positions.

Hashing FuzzyPoW (H-FuzzyPoW). Another way to reduce the computation and communication overhead of the basic FuzzyPoW is to reuse the similarity-preserving hash to develop a H-FuzzyPoW scheme, as shown in Fig. 6. Here, if S wants to verify that C is in possession of a similar ciphertext, it first randomly chooses a number γ and then sends it to C . Once receiving γ , C will compute both $\tilde{k}_{c_w} \leftarrow \text{FKG}(\gamma, c_w)$ and $\{k_{c_w}, P_{c_w}\} = \text{RKG}(1^\lambda, \tilde{k}_{c_w})$ as described in the construction of FuzzyMLE. Then, it calculates $\tilde{h}_{c_w} = h(k_{c_w})$ and sends $\tilde{h}_{c_w}$ with helper information P_{c_w} back to S , where $h(\cdot)$ denotes a collision-resistant hash. After receiving the message from C , S computes $\tilde{k}_{c_{w'}} \leftarrow \text{FKG}(\gamma, c_{w'})$, $k_{c_{w'}} \leftarrow \text{REP}(\tilde{k}_{c_{w'}}, P_{c_w})$, and $\tilde{h}_{c_{w'}} = h(k_{c_{w'}})$. Then, S can verify whether $\tilde{h}_{c_w} = \tilde{h}_{c_{w'}}$. It is obvious that the similarity detection of H-FuzzyPoW depends on the result of adopted similarity-preserving hash algorithm. It will reduce the distance computation overhead from bit-length of a file to that of the hash digest. However, the major limitation is that the protocol cannot determine the precise Hamming distance between different ciphertexts.

4.3 Efficient Hamming Query

Assume the tag of each record is an n -bit binary value generated from a similarity-preserving hash algorithm, we then define the efficient Hamming query below.

Definition 8 (Efficient Hamming Query). Given a collection of N n -bit binary tags and an n -bit binary tag x , an efficient Hamming query can identify any tag y in the collection that $\text{dis}_{\text{Ham}}(x, y) < t$ with less than $O(Nn)$ bit comparison.

We now design both the Hamming distance reduction and tag cutting optimization for efficient tag query.

Hamming Distance Reduction. We observe that it is unnecessary to compute the tag distances with all records of the database in a query. For the sake of explanation, we first define a 1-bit sum function $\text{1-bit}(\cdot)$ (i.e., Hamming weight) and then provide the Hamming distance reduction theorem.

Definition 9 (1-bit). For an n -bit string β , 1-bit sum function $\text{1-bit}(\beta)$ computes 1-bit number (Hamming weight) of string β .

Theorem 1 (Hamming Distance Reduction). For any two n -bit string α and β , if $\text{dis}_{\text{Ham}}(\alpha, \beta) < t$, it must yield $-t < \text{1-bit}(\alpha) - \text{1-bit}(\beta) < t$.

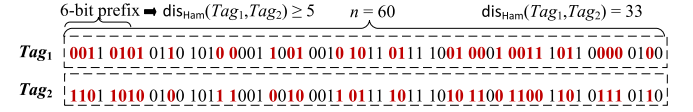


Fig. 7. An example of tag similarity verification.

Proof. (Theorem 1) Without loss of generality, we assume that $\text{1-bit}(\alpha) > \text{1-bit}(\beta)$. Then, we prove the theorem by showing that if $\text{1-bit}(\alpha) - \text{1-bit}(\beta) \geq t$ then we have $\text{dis}_{\text{Ham}}(\alpha, \beta) \geq t$. We simulate the Hamming distance computation process and suppose that we place $\text{1-bit}(\beta)$ 1 bits of α into each 1-bit position of β . Then there will be no 1-bit position left in β while at least t 1 bits left in α . Thus, if the left 1 bits are needed to be placed in some positions of α we have to place them in 0-bit positions. Since the rest bits of α and β are all 0 bits, then all the left 0 bits of α are placed in the 0 bits of β one by one. Finally, if we compute the Hamming distance of any two binary strings α and β , we have $\text{dis}_{\text{Ham}}(\alpha, \beta) \geq t$. Thus, the result follows. $\square$

According to Theorem 1, the cloud server can preprocess the database as shown in Algorithm 3. The server computes the Hamming weight for each database record and sorts data records according to the Hamming weight of tags for Hamming distance reduction in each query.

Algorithm 3. Database Preprocessing Algorithm

Input: An empty database $\mathbb{DB}$; n -bit tag records.

Output: $\mathbb{DB}$ ordered by Hamming weights of records' tags.

Function Preprocess($\mathbb{DB} := \emptyset$)

for each record do

$\text{record.cipher} = c$;

$\text{record.tag} = \tau$;

$\text{record.1bits} = 0$;

$\text{record.1bits} = \text{bit_count}(\text{record.tag})$;

Insert record to $\mathbb{DB}$.

Order $\mathbb{DB}$ by record.1bits in ascending order.

Tag Cutting Optimization. It is observed that, since we only consider the query of tags within small Hamming distance, the tag similarity verification can stop when x -bit prefixes Hamming distance between two tags is equal to or greater than a Hamming distance threshold t . Fig. 7 shows an example of tag similarity verification. There are two 60-bit tags Tag_1 and Tag_2 , and their Hamming distance is $\text{dis}_{\text{Ham}}(\text{Tag}_1, \text{Tag}_2) = 33$. When $t = 5$, tag dissimilarity can be determined by computing the Hamming distance of 6-bit prefixes of the two tags, i.e., $\text{dis}_{\text{Ham}}(\text{Prefix}_{x=6}(\text{Tag}_1), \text{Prefix}_{x=6}(\text{Tag}_2)) = \text{dis}_{\text{Ham}}(001101, 110110) \geq 5$. Here, $\text{Prefix}_x(\cdot)$ denotes the x -bit prefix of a binary string. Ideally, consider the tags of all database records as random binary strings, then $\text{Tag}_\alpha \oplus \text{Tag}_\beta$ is a random binary string. Let X be a discrete random variable defined to be the number of prefix bits chosen by S , P_X be the probability that is less than t 1 bits in $\text{Prefix}_x(\text{Tag}_\alpha) \oplus \text{Prefix}_x(\text{Tag}_\beta)$, we then have

$$P_X = P\{X < t\} = \binom{x}{0} \left(\frac{1}{2}\right)^0 \left(\frac{1}{2}\right)^x + \binom{x}{1} \left(\frac{1}{2}\right)^1 \left(\frac{1}{2}\right)^{x-1} + \cdots + \binom{x}{t-1} \left(\frac{1}{2}\right)^{t-1} \left(\frac{1}{2}\right)^{x-t+1}.$$

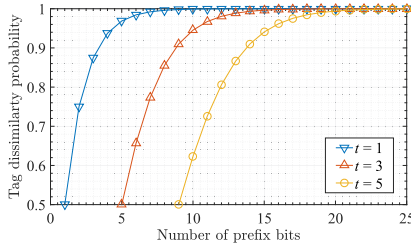


Fig. 8. Verification number of bits in tag equality test.

Thus, $1 - P_X$ indicates the probability of tag dissimilarity, and Fig. 8 plots it for different t and x . Obviously, when t is small, S can detect the dissimilarity of two tags with small prefix length: e.g., if t is 1, 3, 5, then S asks for 5, 11, 16 bits in order to achieve P_X of at least 95%, respectively.

Algorithm 4. Database Query Algorithm

Input: Database $\mathbb{DB}$ with n -bit tag records; n -bit query tag Tag_α ; Hamming distance t .
Output: Database record $record$.
Function Query($\mathbb{DB}$, Tag_α , t , n)
for each $record$ **do**
 $Tag_\beta = record.tag$; $l = 0$; $w = 0$;
 $u = 1_bit(Tag_\alpha)$;
 if $-t < record.bits - u < t$ **then**
 Compute $r_cut(Tag_\alpha)$; Compute $r_cut(Tag_\beta)$;
 for $i = 1$; $i \leq r$; $i++$ **do**
 $l = l + 1_bit(Tag_{\alpha_i} \wedge Tag_{\beta_i})$;
 if $l > t$ **then**
 break;
 else if $i == r$ **then**
 return $record$;

Definition 10 (r -cut [47]). For an n -bit tag Tag_α , r -cut represents r split substrings ($Tag_{\alpha_1}, \dots, Tag_{\alpha_r}$) of Tag_α , the first $r - (n \bmod r)$ substrings are of length $\lceil n/r \rceil$ and the last $n \bmod r$ substrings are of length $\lfloor n/r \rfloor$.

Based on the above observations, we define an r -cut algorithm for tag cutting optimization and apply it to database query in Algorithm 4. With a tag (e.g., Tag_α) as input, the server runs the query algorithm to compute the 1-bit number of Tag_α , i.e., $u = 1_bit(Tag_\alpha)$. Then, it conducts Hamming distance reduction and finds the records with $-t < record.bits - u < t$ as target inputs. For a query tag Tag_α and each record (e.g., Tag_β) in $\mathbb{DB}$, through the r -cut algorithm as defined in Definition 10, the query algorithm cuts them into r sub-strings ($Tag_{\alpha_1}, \dots, Tag_{\alpha_r}$) and ($Tag_{\beta_1}, \dots, Tag_{\beta_r}$), respectively. After that, for the i th loop, the query algorithm computes $dis_{Ham}(Tag_{\alpha_i}, Tag_{\beta_i})$ and adds it with all the previous distances to verify whether it exceeds t . Finally, the algorithm returns a record with tag (e.g., Tag_β) satisfying $dis_{Ham}(Tag_\alpha, Tag_\beta) < t$. If the query algorithm returns one or more records, the server then runs FuzzyPoW with the client for data ownership proof. Otherwise, it means that no similar record is found in the server's database and the client will upload his ciphertext and relative parameters to the server.

main $\mathcal{G}_2$ $\{r_1, r_2\} \leftarrow PG(1^\lambda); b \leftarrow \{0, 1\}; (W, Z) \leftarrow \mathcal{M}(1^\lambda)$ for $i = 1, \dots, W $ do $\tilde{k}_{W[i]} \leftarrow FKG(r_2, W[i])$ $\{k_{W[i]}, P_{W[i]}\} \leftarrow RKG(1^\lambda, \tilde{k}_{W[i]}; r_{W[i]})$ $C_1[i] \leftarrow \text{ENC}(r[i], W[i]) = G(r[i]) \oplus W[i]$ $C_0[i] \leftarrow \{0, 1\}^{C_1[i]}$ $b' \leftarrow \mathcal{A}(r_1, r_2, P_{W[i]}, C_b, Z)$ return $b = b'$	main $\mathcal{G}_3$ $\{r_1, r_2\} \leftarrow PG(1^\lambda); b \leftarrow \{0, 1\}; (W, Z) \leftarrow \mathcal{M}(1^\lambda)$ for $i = 1, \dots, W $ do $\tilde{k}_{W[i]} \leftarrow FKG(r_2, W[i])$ $\{k_{W[i]}, P_{W[i]}\} \leftarrow RKG(1^\lambda, \tilde{k}_{W[i]}; r_{W[i]})$ $C_1[i] \leftarrow \text{ENC}(r[i], W[i]) = f(r[i]) \oplus W[i]$ $C_0[i] \leftarrow \{0, 1\}^{C_1[i]}$ $b' \leftarrow \mathcal{A}(r_1, r_2, P_{W[i]}, C_b, Z)$ return $b = b'$
---	---

Fig. 9. Games used in proof of Theorem 2.

5 PROOF OF SECURITY

Similar to MLE [18], the security of FuzzyMLE is built upon unpredictable data, where plaintexts are drawn from a distribution with negligible guessing probability. We have the following security theorem for FuzzyMLE.

Theorem 2 (Security of the FuzzyMLE). Let SS be an $(\mathcal{M}, m, \tilde{m}, t)$ -secure sketch, and the adopted similarity-preserving hash algorithm has both the similarity and one-wayness properties, then FuzzyMLE with content dependent ciphertext is correct and secure.

Proof. Assume that the data (e.g., w) is unpredictable, and the data owners are honest. The correctness of the FuzzyMLE is obvious. As described in FuzzyDedup, similar plaintexts (w and w') from different clients will lead to similar tags (i.e., τ_w and $\tau_{w'}$) and identical random key k_w . Thus, for data in $\mathcal{M}$, the key k_w output from RKG and $k_{w'}$ from REP, if the inputs are within a certain distance (e.g., $dis_{Ham}(w, w') < t$), we have $DEC(k_{w'}, ENC(k_w, w)) = w$.

For privacy, we use a game playing proof. Let $\mathcal{G}_1$ be identical to Fuzzy-PRV $\$$ -CDA $_{FMLE, \mathcal{M}}^A(\lambda)$.

In $\mathcal{G}_2$, as shown in Fig. 9, each $k_{W[i]}$ is substituted by a totally random $r[i]$ with the same length.

From the security property of $(\mathcal{M}, m, \lambda, t, \epsilon)$ -fuzzy extractor, for any w from $\mathcal{M}$ of min-entropy m , the random key k_w is nearly uniform even for those who observe P_w . More precisely, if $(k_w, P_w) \leftarrow RKG(1^\lambda, \tilde{k}_w)$, then $SD((k_w, P_w), (U_\lambda, P_w)) \leq \epsilon$ [35], where U_λ is a random string with the same length as k_w .

Now, for $|W|$ messages, each has min-entropy m , $SD((\{k_{W[i]}\}_{i=1, \dots, |W|}, \{P_{W[i]}\}_{i=1, \dots, |W|}), (\{r[i]\}_{i=1, \dots, |W|}, \{P_{W[i]}\}_{i=1, \dots, |W|})) \leq |W|\epsilon$, each $r[i]$ is random with the same length as $k_{W[i]}$. Thus, the difference of the advantage between $\mathcal{G}_2$ and $\mathcal{G}_1$ is at most $|W|\epsilon$.

In $\mathcal{G}_3$, the output of G in the encryption of each $W[i]$ is substituted by a totally random one-time pad $R[i]$. Assume the advantage for distinguishing $G(r[i])$ from $R[i]$ is δ , which is negligible due to the pseudorandomness of G , then these substitutions will make a difference in the advantage between $\mathcal{G}_3$ and $\mathcal{G}_2$ at most $|W|\delta$.

Obviously, in $\mathcal{G}_3$, all the adversary obtained are totally random, so its advantage to win the game is $1/2$. So the advantage in $\mathcal{G}_1$, $\text{Adv}_{FMLE, \mathcal{M}, A}^{\text{fuzzy-prv}\$-cda}(\cdot) \leq |W|(\epsilon + \delta)$, which is negligible.

Thus, we have that FuzzyMLE is correct and private, which completes our proof. $\square$

According to the security proof of [40], [48], we have the following security theorems for FuzzyPoWs.

Theorem 3 (Security of the FuzzyPoW). *In OT-hybrid model, if the underlying OT_1^2 scheme is secure in semi-honest model, FuzzyPoW achieves security in the semi-honest setting; if the underlying OT_1^2 is secure in malicious model, FuzzyPoW achieves privacy against a malicious $\mathcal{C}$ and security against a malicious $\mathcal{S}$.*

Proof. *Security against the semi-honest adversary.* If both $\mathcal{C}$ and $\mathcal{S}$ behave honestly and the adopted OT protocol is secure, the outputs of the OT give no information on the inputs of the $\mathcal{C}$ and $\mathcal{S}$. Thus, $\mathcal{C}$ cannot get any information about the input (i.e., c_w and $\text{dis}_{\text{Ham}}(c_w, c_{w'})$) of $\mathcal{S}$ through the protocol. $\mathcal{S}$ can only get the blinded bit values of $c_{w'}$, the sum of numbers randomly selected by $\mathcal{C}$ and the Hamming distance $\text{dis}_{\text{Ham}}(c_w, c_{w'})$. Thus, FuzzyPoW is secure against the semi-honest adversary.

Privacy Against Malicious $\mathcal{C}$. Assume that the adopted OT protocol is secure in the malicious model and there is a malicious adversary $\mathcal{A}$ controlling $\mathcal{C}$. Since no messages are sent to $\mathcal{C}$, the views of $\mathcal{A}$ are computationally indistinguishable between the cases where $\mathcal{S}$ inputs y and y' as defined in Equation (2). Thus, the basic FuzzyPoW protocol is privacy against a malicious $\mathcal{C}$.

Security Against Malicious $\mathcal{S}$. Let $\mathcal{A}$ be a PPT adversary controlling $\mathcal{S}$ in the real world, we describe a simulator $\mathcal{S}_{\mathcal{A}}$ who simulates the view of $\mathcal{A}$ in the ideal world. $\mathcal{S}_{\mathcal{A}}$ runs $\mathcal{A}$ on input c_w . Since we operate in the OT-hybrid model, $\mathcal{A}$ can change input and send $\hat{c}_w = (c_w[1], \dots, c_w[n])$ to the OT oracle. $\mathcal{S}_{\mathcal{A}}$ sends $\hat{c}_w$ to the trusted party and obtains $D = \text{dis}_{\text{Ham}}(c_w', \hat{c}_w)$. $\mathcal{S}_{\mathcal{A}}$ picks n random values $\{t_1, \dots, t_{n-1}, R\} \in \mathbb{Z}_{n+1}$ and computes $t_n = R + D - \sum_{i=1}^{n-1} t_i$. $\mathcal{S}_{\mathcal{A}}$ sends the t_i s to $\mathcal{A}$ as results of the oblivious transfer. $\mathcal{S}_{\mathcal{A}}$ then sends R to $\mathcal{A}$ and outputs whatever $\mathcal{A}$ outputs. Now, we can prove the indistinguishability of the protocol between the real and the ideal executions. Let v be a random subset of size D of $\{1, \dots, n\}$. It represents the bit positions where $c_w[i] \oplus c_w'[i] = 1$.

We have to prove that $\mathcal{A}$ cannot distinguish between his view during the real execution and ideal execution through interaction with $\mathcal{S}_{\mathcal{A}}$. Consider the following two distributions:

- Real execution distribution D_v : Choose n uniformly random values $\{r_1, \dots, r_n\} \in \mathbb{Z}_{n+1}$. For each $i \in \{1, \dots, n\}$, compute $t_i = r_i + 1$ if $i \in v$ and let $t_i = r_i$ otherwise. Output $(t_1, \dots, t_n)$.
- Ideal execution distribution D'_v : Choose n uniformly random values $\{t'_1, \dots, t'_{n-1}, R\} \in \mathbb{Z}_{n+1}$. Compute $T = R + D$ and $t'_n = T - \sum_{i=1}^{n-1} t'_i$. Output $(t'_1, \dots, t'_n)$.

Based on the above description, it is very intuitive that D_v and D'_v are identically distributed and the sampling from D'_v only requires the knowledge of v . More precisely, the distribution D_v represents the view of $\mathcal{A}$ in a real execution of FuzzyPoW. With the only knowledge of the final output, the simulator $\mathcal{S}_{\mathcal{A}}$ can sample from D'_v . Therefore, the view of $\mathcal{S}$ in the real world and the simulated view of $\mathcal{S}$ in the ideal world are indistinguishable, which ensures that FuzzyPoW is full security against a malicious $\mathcal{S}$.¹ $\square$

Theorem 4 (Security of the P-FuzzyPoW). *In OT-hybrid model, if the underlying OT_1^2 scheme is secure in semi-honest model, P-FuzzyPoW achieves security in the semi-honest setting; if the underlying OT_1^2 is secure in malicious model, P-FuzzyPoW achieves privacy against a malicious client $\mathcal{C}$ and a malicious $\mathcal{S}$.*

Proof. (Theorem 4) Intuitively, P-FuzzyPoW is designed to efficiently compute the common sampling bits for ownership proof, which is independent of the value of the selected bits. The difference between the FuzzyPoW and P-FuzzyPoW is the random sampling phase, which will not provide additional information about the inputs of both $\mathcal{C}$ and $\mathcal{S}$. Thus, according to Theorem 3, the P-FuzzyPoW protocol achieves privacy against a malicious $\mathcal{C}$ and security against a malicious $\mathcal{S}$. $\square$

Theorem 5 (Security of the H-FuzzyPoW). *If the adopted similarity-preserving hash algorithm provides similar digest and it is difficult for an adversary to compute the pre-image of a digest, then H-FuzzyPoW is secure.*

Proof. (Theorem 5) Intuitively, we assume that the cloud users' plaintexts drawn from a distribution with negligible guessing probability and the adopt similarity-preserving hash provide both digest similarity property and one-way property. H-FuzzyPoW provides ownership proof through the digest similarity property where only the cloud server and users with similar ciphertext will be able to compute similar digest. Its security is base on the one-way property of similarity-preserving hash. When $\mathcal{C}$ is malicious with $\tau_{w'} = \text{TAG}(r_1, w)$ while not w , it is difficult for the client to compute $\hat{h}_{c_w}$ where $\hat{h}_{c_w} = \hat{h}_{c_{w'}}$. When receiving $\hat{h}_{c_w}$, it is also difficult for the cloud server to compute the w . That is to say, by executing H-FuzzyPoW, the cloud server is difficult to compute the plaintext of the client and the client cannot cheat the server by computing a similar verification digest without a similar plaintext. Thus, H-FuzzyPoW is secure. $\square$

The efficiency enhancement variants of FuzzyPoW ask $\mathcal{C}$ to prove to $\mathcal{S}$ that the Hamming distance between its input c'_w and $\mathcal{S}$'s input c_w is smaller than a Hamming distance threshold T in a secure way. According to Theorems 2, 3, 4 and 5, we have the following theorem:

Theorem 6 (Secure Hamming Distance Measurability). *If FuzzyMLEs and FuzzyPoWs are secure, FuzzyDedup is efficient in computation, storage and communication, while achieving both Hamming distance measurability and similarity security.*

Proof. (Theorem 6) On the one hand, FuzzyMLE can encrypt users' data before sending to the cloud server and allows the server to conduct secure similar data deduplication. Thus, the system is storage efficiency. In FuzzyDedup, if the cloud server has ever stored a ciphertext c_w , a subsequent user, with w' whose ciphertext $c_{w'}$ is similar to c_w (i.e., $\text{dis}_{\text{Ham}}(c_{w'}, c_w)$), does not need to upload $c_{w'}$. Therefore, we have that the system is communication efficiency. On the other hand, FuzzyPoWs can securely provide exact, probabilistic or hashing Hamming distance results for $\mathcal{S}$ according to Theorems 3, 4 and 5. Since

1. The proof of Theorem 3 is mainly based on the proofs in [40].

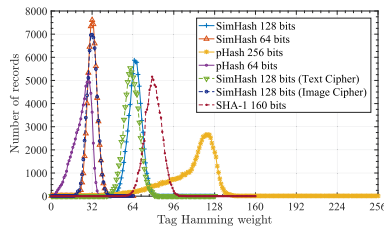


Fig. 10. Tag distribution.

the FuzzyMLEs and the FuzzyPoWs do not introduce heavy computation algorithm and they will finish in polynomial time, our system is also efficient in computation. Finally, the cloud server can conduction similar ciphertext deduplication based on the computed distance in FuzzyDedup and both the cloud server and cloud users cannot obtain additional information about the other parties' input. Therefore, FuzzyDedup provides both Hamming distance measurability property and similarity property. □

6 PERFORMANCE EVALUATIONS

In this section, we present the implementation of FuzzyDedup and conduct experiments to evaluate its performance.

6.1 System Implementation

System Setting. FuzzyDedup is implemented with approximately 3,000 lines of C++ code and employs MySQL database system as the backend data storage system. Free GMP library [49] and OpenCV library [50] are called to implement the SimHash and pHash, respectively. OpenSSL library [51] is called to implement the cryptographic hash and XOR encryption algorithms (i.e., SHA-1, SHA-256 and XOR cipher). We simulate the clients and a server on two separate machines, both of them are running Ubuntu 16.04 LTS, with 3.10-GHz Intel Core i9-9900 CPU and 16 GB of memory. The communication bandwidth for the wired connection between the server and a client is set to be 100 Mbps. To achieve the remote data operation between the client and server, we use sockets to implement the Remote Procedure Call (RPC).

Datasets. In our experiments, real-world datasets are considered, e.g., TREC public spam corpus [52] and keyframes of CC_WEB_VIDEO [53], where the first one consists of 75,000 text files and the length of each one is between 1 KB to 30 KB while the second one includes more than 500,000 images. We sample the data records from the two datasets and the number of data records will vary in the respective performance studies, ranging from 10,000 to 100,000. Besides, we also take the Stanford dogs dataset, containing 20,580 images from ImageNet dataset, to evaluate the effectiveness and efficiency of FuzzyDedup and its counterparts.

6.2 Evaluations

Tag Query Time Cost. Note that the Hamming weight of tags from the whole dataset is important to determine the tag query time cost. We first analyze the distributions of Hamming weights and tags so as to exhibit their statistic property over the chosen dataset. We conduct our experiments to perform similar data deduplication over 75,000 non-

duplicate E-mail files from TREC07p dataset and 76,000 images from CC_WEB_VIDEO dataset, respectively. By extracting all the tags in the server-side database, we plot the distributions of number of tags related to Hamming weights in each dataset and present them in Fig. 10. It is seen that, both the distribution of 128-bit SimHash tags and 64-bit SimHash tags follow the Gaussian distribution, and their expectations are nearly half of the tag length. Both the number of 64-bit pHash tags and 256-bit pHash tags follow a nearly Gaussian distribution over Hamming weights, while their expectations are relatively smaller than the half of the tag length. Through the distribution of pHash tags, we further check the content of the corresponding images and find that the white color is more likely to generate 0 bits in pHash tag, while the black color is on the opposite. Inspired by this property, we believe that the distribution of images will become closer to Gaussian distribution if the tag of each image is computed from its ciphertext. Therefore, we suggest using the ciphertext of both texts and images as the inputs to compute the file tags. The number of SimHash tag with respect to the texts and images follow more decent Gaussian distributions over their Hamming weights. For comparison, we also plot the number of SHA-1 tag distribution over Hamming weight, which is similar to those distributions of text ciphertexts and image ciphertexts. We need to remark that, pHash relies on the Discrete Cosine Transform (DCT), which discards the coefficients of high frequencies and retains the low frequencies. Thus, when we apply pHash on the encrypted ciphertext, the Hamming weights of most tags become less than 5. Therefore, we further consider encrypted images as text strings with 80-bit words in our experiments and apply SimHash for the generation of their tags as plotted in the figure.

In FuzzyDedup, the similarity duplication detection time cost of each file depends on the number of database records to be compared and the computation time cost of each comparison. To achieve the system level deduplication optimization, we set tag cutting optimization with $r = 3$ in our simulation, according to the results in [47]. We then compare the query time costs for the cases with and without Hamming distance reduction. The results are shown in Fig. 12, which indicate that, in both cases (Figs. 12a and 12b), the query time costs increase with the number of records while a larger Hamming distance t results in longer query time. Notably, for the same number of records in Figs. 12a and 12b, the time cost from our optimization will reduce to less than 5, 10, 15 percent compared to that in the query without Hamming distance reduction when $t = 1$, $t = 3$, $t = 5$, respectively.

Finally, we compare the tag query time costs in the basic, server-aided, and random-tag FuzzyMLE schemes, respectively, when the Hamming distance $t = 5$ and the sub-string tag number $r = 3$. The results are shown in Fig. 11. Obviously, the tag query time cost grows with the number of records in all three schemes. When querying over the same number of records, the basic scheme and the random-tag scheme incur the lowest and highest tag query time cost, respectively, among the three schemes.

Impact of Various Parameters on Deduplication Effectiveness. Both the tag distance and ciphertext distance will highly impact the deduplication effectiveness. Hence, it is necessary

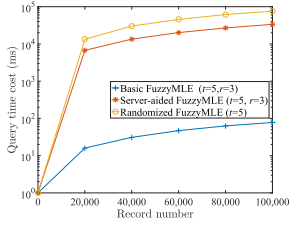
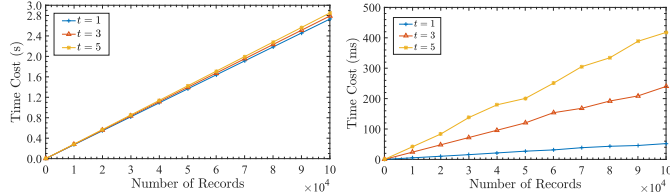


Fig. 11. Scheme compare.



(a) Without Hamming reduction (b) With Hamming reduction

Fig. 12. Query time cost comparison.

to measure the relationship between the tag similarity and data similarity for assisting the system to set the proper data similarity parameters. To achieve this goal, we elaborately select a subset of identical E-mail text files from TREC07p dataset (i.e., 19,736 files that size ranging from 1 KB to 30 KB) and generate different datasets by randomly substituting the identical number of words (i.e., from 1 letter to 12 letters) of each file in each dataset. Then, we evaluate the deduplication effectiveness related to the tags' Hamming distance and file similarity, with the results shown in Fig. 13a. We can see, for specific tag Hamming distance, the ratio of duplicate files decreases with the increase of file dissimilarity. On the other hand, for the specific file similarity, the ratio of duplicate files increases with the growing of tag Hamming distance. Thus, a smaller t is expected to deduplicate files with shorter distance. For example, only about 70% of the files in our dataset are considered similar, when the tag Hamming distance threshold t is set to 2 and only 1 letter of each file is changed. However, the ratio of duplication will raise to over 90% if $t = 6$ and changing 2 letters of each file.

Similarly, we also select 22,317 images with the size of 320×240 pixels from keyframes of CC_WEB_VIDEO dataset and change a specific fraction of pixels of the dataset for evaluation. Fig. 13b reports the relationship among image similarity, tag distance and ratio of duplicate file detection. It is seen that both larger ratio of randomly modified pixels and smaller tag Hamming distance will lead to a smaller ratio of duplicate detection. For instance, if 1.5% pixels of each file is changed in the dataset and t is set to 1, only about 30% images are detected as similar duplicates.

Comparing to the Chunk-Based Deduplication. We next compare FuzzyDedup to the chunk-based deduplication [24], in terms of the deduplication ratio, defined as

$$\frac{\text{original data size}}{\text{data size after deduplication}}.$$

We take the Stanford dogs

dataset (20,580 images) from ImageNet dataset for experiments and generate three near-duplicates for each image by adding different numbers of random noise points, ranging from 0 to 100. Fig. 14 shows the deduplication ratios of chunk-based deduplication scheme and our FuzzyDedup.

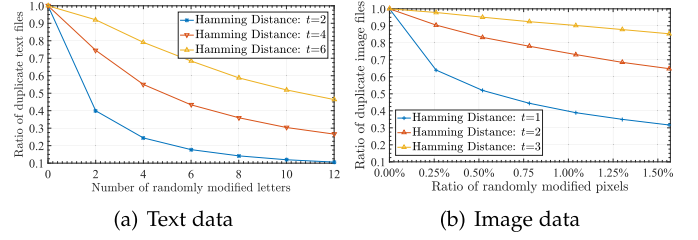


Fig. 13. Deduplication ratio with data similarity and tag distances.

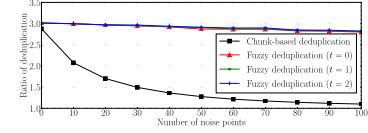


Fig. 14. Dedup. ratios comparison.

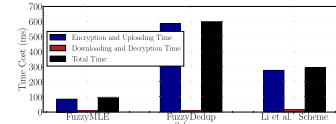


Fig. 15. Efficiency comparison.

Notably, for our FuzzyDedup, we consider three different hamming distances, i.e., $t = 0, 1$ and 2 . We observe that our FuzzyDedup can achieve the similar deduplication ratio as the chunk-based scheme, when $t = 0$, for identical deduplication. But when we set $t = 1$ and 2 , our FuzzyDedup can achieve much higher deduplication ratio than the chunk-based scheme and such a ratio grows with the increasing of noise points.

Comparing to the Existing Fuzzy Deduplication Scheme [27]. We next evaluate the efficiency of FuzzyDedup by comparing to the existing fuzzy deduplication scheme (Li *et al.*'s scheme [27]) with respect to data outsourcing procedure (file encryption and uploading to server's database) and fetch procedure (file downloading and decryption) in the LAN setting. Here, the time cost is evaluated when a 400 KB image is outsourced to (or fetched from) the server database storing 20,000 non-replicated records. The Hamming Distance of hash digest is set to 1 for tag similarity detection in our evaluation. Fig. 15 presents the maximum time costs corresponding to the outsourcing and fetching process from FuzzyMLE, FuzzyDedup, and Li *et al.*'s scheme [27]. In this figure, the outsourcing and fetching processes of our FuzzyDedup take 590 ms and 12 ms, respectively, while Li *et al.*'s scheme [27] takes 278 ms and 15 ms, respectively. Compared to Li *et al.*'s scheme, our FuzzyDedup needs more time in the outsourcing process, but notably, the former one requires a trust manager to manage all keys. With the milli-second-level time cost, we believe our FuzzyDedup is practical for use.

7 RELATED WORKS

Convergent Encryption (CE) was the first secure deduplication solution proposed by Douceur *et al.* [17], in which each client encrypts its plaintext with the deterministic symmetric encryption scheme where the key is derived by computing the deterministic hash of the plaintext. Through this

way, the same plaintext will arrive at the same ciphertext, which makes deduplication possible. Later, Bellare *et al.* [18], for the first time, generalized the framework of CE by proposing a novel Message-Locked Encryption (MLE) scheme while demonstrating the practicality and theoretical soundness of their design. To enhance security level, Abadi *et al.* [20] designed two alternative schemes, namely fully randomized scheme and determined scheme, to further strengthen the security of lock-dependent schemes. To provide security against off-line brute-force attacks, Bellare *et al.* [19] proposed a server-aided secure deduplication scheme, called DupLESS, through involving an additional key server. Later, Duan [54] proposed a distributed key generation protocol and Miao *et al.* [55] proposed a multiple server-aided DupLESS to overcome the single point failure of the key server. To avoid the dependency on additional independent servers, Liu *et al.* [21] presented the first single server deduplication scheme that is secure against online brute-force attacks from compromised clients or server without the aid of an independent server. In addition, Bellare and Keelveedhi [56] extended the MLE to support semantically secure deduplication of correlated and public system parameters dependent data in an interactive manner, which they called interactive MLE (iMLE), by exploiting fully homomorphic encryption. To achieve efficient and reliable key management, Li *et al.* [57] considered the management of convergent keys and designed a Dekey scheme based on threshold secret sharing technique. Later, Li *et al.* [58] designed a scheme based on all-or-nothing transform, which realized efficient re-keying in encrypted deduplication storage systems. More recently, various flavors of secure cloud data deduplication schemes are designed to support different functionalities or application scenarios, such as mobile crowdsensing [59], scalable access control [60], active key update [61] and side-channel attacks security [62]. In data ownership proof, Halevi *et al.* [36] figured out that, without user identity verification, an adversary with a very small fingerprint generated from data can convince the cloud server that he/she owns the data for downloading the data from the cloud. To mitigate such a threat, Proof of oWnership (PoW) schemes [36] based on different verification structures are introduced to verify clients' ownership of particular data with challenge/response protocols. Later, various schemes are designed to improve PoW schemes in different aspects such as efficiency [63], security [64], and functionalities [65].

However, all aforementioned works, focusing on the secure identical data deduplication [18], [19], [20], [56], [66], cannot be applied to the similar data. Duplicate Cleaner [10], VisiPics [11], and dupeGuru [12] have been developed for providing the functionality of fuzzy similar data deduplication, but only in the plaintext domain. Although the chunk-based schemes [23], [24], [25], [26] have been designed, which can be used to deduplicate similar files either in the plaintext or ciphertext domains, the nature of these solutions is to deduplicate identical trunks not similar ones, inapplicable to similar data deduplication (like the similar trunk). Also, determining the suitable-sized chunk for achieving the effective deduplication is extremely hard. On the other hand, Li *et al.* [27] has designed a secure fuzzy image deduplication scheme based on perceptual hash and

cryptographic hash for group users with shared group keys. But, it relies on a trusted entity to manage the keys for all group users and can not work among different groups. Later, proof of retrieval and ownership is explored by leveraging error-correcting code and Merkle hash tree in [28] based on Li *et al.*'s scheme, which also inherited the limitations of the previous scheme. Takeshita *et al.* [29] designed another secure nearly-identical image deduplication scheme and PoW based on access control with one single server. The scheme is based on multiple secure locality-sensitive hash (LSH) and password-authenticated key exchange (PAKE). With PAKE, the current cloud user, with the help from the cloud server, needs to wait on line for the assistance from the previous file owner when conducting deduplication of a file. The limitation lies in whether the previous data owners can always stay online to help the upcoming cloud users. In sharp contrast to these work, our FuzzyDedup does not rely on the help from a trust key manager or online cloud users, which is more practical and scalable in the public cloud scenario.

8 CONCLUSION

This paper designed the first secure fuzzy deduplication system FuzzyDedup to enable a cloud platform to conduct the client-side deduplication and eliminate ciphertexts with similar plaintext. Two key components were designed in FuzzyDedup, i.e., FuzzyMLE and FuzzyPoW, which can achieve the ciphertext-based deduplication for similar data and allow a server to verify the possession of similar data from a client, respectively. In addition, the server-aided key generation and tag randomization were integrated to further enhance the security of FuzzyMLE, while random sampling and similarity-preserving Hash-based FuzzyPow were designed to improve the efficiency of data ownership proof. Moreover, a new query statement of MySQL was designed based on Hamming distance reduction and tag cutting optimization, which can greatly reduce the time cost of the Hamming query by eliminating a large proportion of dissimilar records. In the end, we demonstrated the security of FuzzyDedup through formal security proof and designed various experiments to exhibit both the statistical properties and deduplication effectiveness of FuzzyDedup on real-world text and image datasets.

Our current design focused on data similarity with respect to the Hamming distance, but it is necessary yet challenging to design solutions that support similarity measurement of other distances, including but are not limited to Levenshtein distance, Cosine distance, and others. On the other hand, a similarity-preserving Hash or related solution that is more efficient and cryptographically secure in similar data identification is also desirable. We will leave the exploration of these open problems in our future work. Furthermore, since our FuzzyDedup employs a set of advanced technologies which has higher computational complexity, comparing the existing identification deduplication solutions. In practical deployment, we can first apply the identical deduplication schemes (such as MLE-style) to deduplicate the identical data and then apply our FuzzyDedup to solely deduplicate the similar data, to further improve the efficiency.

REFERENCES

- [1] M. Fröbeet *et al.*, "CopyCat: Near-duplicates within and between the ClueWeb and the common crawl," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 2398–2404.
- [2] R. Ananthakrishna, S. Chaudhuri, and V. Ganti, "Eliminating fuzzy duplicates in data warehouses," in *Proc. 28th Int. Conf. Very Large Data Bases*, 2002, pp. 586–597.
- [3] H. H. Shahri and A. A. Barforoush, "A flexible fuzzy expert system for fuzzy duplicate elimination in data cleaning," in *Proc. Int. Conf. Database Expert Syst. Appl.*, 2004, pp. 161–170.
- [4] F. Geerts, G. Mecca, P. Papotti, and D. Santoro, "The LLUNATIC data-cleaning framework," *Proc. VLDB Endowment*, vol. 6, no. 9, pp. 625–636, 2013.
- [5] T. Pönitz and J. Stöttinger, "Efficient and robust near-duplicate detection in large and growing image data-sets," in *Proc. 18th Int. Conf. Multimedia*, 2010, pp. 1517–1518.
- [6] J. Liu, Z. Huang, H. Cheng, Y. Chen, H. T. Shen, and Y. Zhang, "Presenting diverse location views with real-time near-duplicate photo elimination," in *Proc. IEEE 29th Int. Conf. Data Eng.*, 2013, pp. 505–516.
- [7] D. Schnitzer, A. Flexer, and G. Widmer, "A fast audio similarity retrieval method for millions of music tracks," *Multimedia Tools Appl.*, vol. 58, no. 1, pp. 23–40, 2012.
- [8] X. Wu, A. G. Hauptmann, and C. Ngo, "Practical elimination of near-duplicates from web video search," in *Proc. 15th Int. Conf. Multimedia*, 2007, pp. 218–227.
- [9] Y. Hao, T. Mu, R. Hong, M. Wang, N. An, and J. Y. Goulermas, "Stochastic multiview hashing for large-scale near-duplicate video retrieval," *IEEE Trans. Multimedia*, vol. 19, no. 1, pp. 1–14, Jan. 2017.
- [10] D. S. Ltd., "Duplicate cleaner," 2017. [Online]. Available: <https://www.digitalvolcano.co.uk/duplicatecleaner.html>
- [11] G. Fouet, "Visipics," 2015. [Online]. Available: http://www.visipics.info/index.php?title=Main_Page
- [12] H. Software, "dupeguru," 2016. [Online]. Available: <https://www.hardcoded.net/dupeguru/>
- [13] J. Gantz and D. Reinsel, "The Digital Universe 2020," 2012. [Online]. Available: <https://www.emc.com/collateral/analyst-reports/idc-the-digital-universe-in-2020.pdf>
- [14] D. Bhagwat, K. Eshghi, D. D. E. Long, and M. Lillibridge, "Extreme binning: Scalable, parallel deduplication for chunk-based file backup," in *Proc. IEEE/ACM Int. Symp. Model. Anal. Simul. Comput. Telecommun. Syst.*, 2009, pp. 1–9.
- [15] B. K. Debnath, S. Sengupta, and J. Li, "ChunkStash: Speeding up inline storage deduplication using flash memory," in *Proc. USENIX Annu. Tech. Conf.*, 2010, Art. no. 16.
- [16] K. Srinivasan, T. Bisson, G. R. Goodson, and K. Voruganti, "iDedup: Latency-aware, inline data deduplication for primary storage," in *Proc. 10th USENIX Conf. File Storage Technol.*, 2012, Art. no. 24.
- [17] J. R. Douceur, A. Adya, W. J. Bolosky, D. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proc. 22nd Int. Conf. Distrib. Comput. Syst.*, 2002, pp. 617–624.
- [18] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2013, pp. 296–312.
- [19] S. Keelveedhi, M. Bellare, and T. Ristenpart, "DupLESS: Server-aided encryption for deduplicated storage," in *Proc. 22nd USENIX Conf. Secur.*, 2013, pp. 179–194.
- [20] M. Abadi, D. Boneh, I. Mironov, A. Raghunathan, and G. Segev, "Message-locked encryption for lock-dependent messages," in *Proc. Annu. Cryptol. Conf.*, 2013, pp. 374–391.
- [21] J. Liu, N. Asokan, and B. Pinkas, "Secure deduplication of encrypted data without additional independent servers," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 874–885.
- [22] R. D. Pietro and A. Sorniotti, "Proof of ownership for deduplication systems: A secure, scalable, and efficient solution," *Comput. Commun.*, vol. 82, pp. 71–82, 2016.
- [23] D. Bhagwat, K. Eshghi, D. D. E. Long, and M. Lillibridge, "Extreme binning: Scalable, parallel deduplication for chunk-based file backup," in *Proc. IEEE/ACM Int. Symp. Model. Anal. Simul. Comput. Telecommun. Syst.*, 2009, pp. 1–9.
- [24] M. Lillibridge, K. Eshghi, and D. Bhagwat, "Improving restore speed for backup systems that use inline chunk-based deduplication," in *Proc. USENIX Conf. File Storage Technol.*, 2013, pp. 183–198.
- [25] Y. Zhang *et al.*, "A fast asymmetric extremum content defined chunking algorithm for data deduplication in backup storage systems," *IEEE Trans. Comput.*, vol. 66, no. 2, pp. 199–211, Feb. 2017.
- [26] W. Sun, N. Zhang, W. Lou, and Y. T. Hou, "Tapping the potential: Secure chunk-based deduplication of encrypted data for cloud backup," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2018, pp. 1–9.
- [27] X. Li, J. Li, and F. Huang, "A secure cloud storage system supporting privacy-preserving fuzzy deduplication," *Soft Comput.*, vol. 20, no. 4, pp. 1437–1448, 2016.
- [28] S. P. Bini and S. Abirami, "Proof of retrieval and ownership for secure fuzzy deduplication of multimedia data," in *Proc. Prog. Comput. Analytics Netw.*, 2018, pp. 245–255.
- [29] J. Takeshita, R. Karl, and T. Jung, "Secure single-server nearly-identical image deduplication," in *Proc. 29th Int. Conf. Comput. Commun. Netw.*, 2020, pp. 1–6.
- [30] M. O. Rabin, "How to exchange secrets with oblivious transfer," *IACR Cryptol. ePrint Arch.*, vol. 2005, 2005, Art. no. 187.
- [31] S. Even, O. Goldreich, and A. Lempel, "A randomized protocol for signing contracts," in *Proc. Annu. Cryptol. Conf.*, 1982, pp. 205–210.
- [32] G. Asharov, Y. Lindell, T. Schneider, and M. Zohner, "More efficient oblivious transfer extensions," *J. Cryptol.*, vol. 30, no. 3, pp. 805–858, 2017.
- [33] M. Naor and B. Pinkas, "Efficient oblivious transfer protocols," in *Proc. 12th Annu. Symp. Discrete Algorithms*, 2001, pp. 448–457.
- [34] C. Peikert, V. Vaikuntanathan, and B. Waters, "A framework for efficient and composable oblivious transfer," in *Proc. Annu. Cryptol. Conf.*, 2008, pp. 554–571.
- [35] Y. Dodis, L. Reyzin, and A. D. Smith, "Fuzzy extractors: How to generate strong keys from biometrics and other noisy data," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2004, pp. 523–540.
- [36] S. Halevi, D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Proofs of ownership in remote storage systems," in *Proc. 18th ACM Conf. Comput. Commun. Secur.*, 2011, pp. 491–500.
- [37] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. Annu. Cryptol. Conf.*, 1987, pp. 369–378.
- [38] C. Hazay and Y. Lindell, Eds., *Efficient Secure Two-Party Protocols: Techniques and Constructions*. New York, NY, USA: Springer Science & Business Media, 2010.
- [39] Y. Lindell and B. Pinkas, "An efficient protocol for secure two-party computation in the presence of malicious adversaries," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2007, pp. 52–78.
- [40] J. Bringer, H. Chabanne, and A. Patey, "SHADE: Secure hamming distance computation from oblivious transfer," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2013, pp. 164–176.
- [41] M. Bellare, A. Desai, E. Jorjipii, and P. Rogaway, "A concrete security treatment of symmetric encryption," in *Proc. 38th Annu. Symp. Found. Comput. Sci.*, 1997, pp. 394–403.
- [42] S. Goldwasser and S. Micali, "Probabilistic encryption," *J. Comput. Syst. Sci.*, vol. 28, no. 2, pp. 270–299, 1984.
- [43] D. Chaum, "Blind signatures for untraceable payments," in *Proc. Annu. Cryptol. Conf.*, 1982, pp. 199–203.
- [44] M. Bellare, C. Namprempre, D. Pointcheval, and M. Semanko, "The one-more-RSA-inversion problems and the security of chaum's blind signature scheme," *J. Cryptol.*, vol. 16, no. 3, pp. 185–215, 2003.
- [45] J. Camenisch, A. D. Caro, E. Ghosh, and A. Sorniotti, "Oblivious PRF on committed vector inputs and application to deduplication of encrypted data," in *Proc. 23rd Int. Conf. Financial Cryptogr. Data Secur.*, 2019, pp. 337–356.
- [46] R. Canetti, "Towards realizing random oracles: Hash functions that hide all partial information," in *Proc. Annu. Cryptol. Conf.*, 1997, pp. 455–469.
- [47] A. X. Liu, K. Shen, and E. Torng, "Large scale hamming distance query processing," in *Proc. IEEE 27th Int. Conf. Data Eng.*, 2011, pp. 553–564.
- [48] P. Mohassel, S. Niksefat, S. S. Sadeghian, and B. Sadeghiyan, "An efficient protocol for oblivious DFA evaluation and applications," in *Proc. Cryptogr. Track RSA Conf.*, 2012, pp. 398–415.
- [49] GMP, "The GNU multiple precision arithmetic library," 2016. [Online]. Available: <https://gmplib.org/>
- [50] O. Team, "OpenCV," 2020. [Online]. Available: <https://opencv.org/>
- [51] OpenSSL, "dupeguru," 2016. [Online]. Available: <https://openssl.org/>

- [52] NIST, "Trec public spam corpus," 2007. [Online]. Available: <https://trec.nist.gov/data/spam.html>
- [53] X. Wu, C.-W. Ngo, and A. G. Hauptmann, "CC_web_video: Near-duplicate web video dataset," 2007. [Online]. Available: <http://vireo.cs.cityu.edu.hk/webvideo/>
- [54] Y. Duan, "Distributed key generation for encrypted deduplication: Achieving the strongest privacy," in *Proc. 6th ACM Workshop Cloud Comput. Secur.*, 2014, pp. 57–68.
- [55] M. Miao, J. Wang, H. Li, and X. Chen, "Secure multi-server-aided data deduplication in cloud computing," *Pervasive Mobile Comput.*, vol. 24, pp. 129–137, 2015.
- [56] M. Bellare and S. Keelveedhi, "Interactive message-locked encryption and secure deduplication," in *Proc. IACR Int. Workshop Public Key Cryptogr.*, 2015, pp. 516–538.
- [57] J. Li, X. Chen, M. Li, J. Li, P. P. C. Lee, and W. Lou, "Secure deduplication with efficient and reliable convergent key management," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 6, pp. 1615–1625, Jun. 2014.
- [58] J. Li, C. Qin, P. P. C. Lee, and J. Li, "Rekeying for encrypted deduplication storage," in *Proc. 46th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2016, pp. 618–629.
- [59] J. Ni, K. Zhang, Y. Yu, X. Lin, and X. S. Shen, "Providing task allocation and secure deduplication for mobile crowdsensing via fog computing," *IEEE Trans. Dependable Secur. Comput.*, vol. 17, no. 3, pp. 581–594, May./Jun. 2020.
- [60] R. Xu, J. Joshi, and P. Krishnamurthy, "An integrated privacy preserving attribute-based access control framework supporting secure deduplication," *IEEE Trans. Dependable Secur. Comput.*, vol. 18, no. 2, pp. 706–721, Mar./Apr. 2021.
- [61] J. Li, T. Li, Z. Liu, and X. Chen, "Secure deduplication system with active key update and its application in IoT," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 6, pp. 69:1–69:21, 2019.
- [62] Y. Zhang, Y. Mao, M. Xu, F. Xu, and S. Zhong, "Towards thwarting template side-channel attacks in secure cloud deduplications," *IEEE Trans. Dependable Secur. Comput.*, vol. 18, no. 3, pp. 1008–1018, May/Jun. 2021.
- [63] J. B. Alis, R. D. Pietro, A. Orfila, and A. Sorniotti, "A tunable proof of ownership scheme for deduplication using bloom filters," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2014, pp. 481–489.
- [64] Y. Zhao and S. S. M. Chow, "Towards proofs of ownership beyond bounded leakage," in *Proc. Int. Conf. Provable Secur.*, 2016, pp. 340–350.
- [65] S. Jiang, T. Jiang, and L. Wang, "Secure and efficient cloud data deduplication with ownership management," *IEEE Trans. Serv. Comput.*, vol. 13, no. 6, pp. 1152–1165, Nov./Dec. 2020.
- [66] T. Jiang, X. Chen, Q. Wu, J. Ma, W. Susilo, and W. Lou, "Towards efficient fully randomized message-locked encryption," in *Proc. Australas. Conf. Inf. Secur. Privacy*, 2016, pp. 361–375.



Tao Jiang (Member, IEEE) received the PhD degree from the School of Telecommunications Engineering, Xidian University, Xi'an, China, in 2016. He is now a lecturer with the School of Cyber Engineering, Xidian University. His research interests include applied cryptography and cloud computing security.



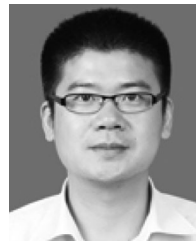
Xu Yuan (Senior Member, IEEE) received the BS degree from the College of Information Technology, Nankai University, Tianjin, China, in 2009, and the PhD degree from the Bradley Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, Virginia, in 2016. From 2016 to 2017, he was a postdoctoral fellow of electrical and computer engineering with the University of Toronto, Toronto, ON, Canada. He is currently an Assistant Professor with the School of Computing and Informatics, University of Louisiana at Lafayette, Lafayette, Louisiana. His research interest focuses on cloud computing, social network security, and cognitive radio networks.



Yuan Chen received the PhD degree from the School of Telecommunication Engineering, Xidian University, Xi'an, China, in 2006. She is currently an associate professor in State Key Laboratory of Integrated Service Networks (ISN), Xidian University, China. Her research interest is in cryptography, and mainly focuses on design and applications of Public-Key Encryptions, design and analysis of cryptographic protocols, and anonymous communication.



Ke Cheng received the BS and MS degrees from Anhui University, China, in 2015 and 2018, respectively. He is currently working toward the PhD degree with the School of Computer Science and Technology, Xidian University, China. His research interests include data security and privacy protection technology.



Liangmin Wang (Member, IEEE) received the BS degree in computational mathematics from Jilin University, Changchun, China, in 1999, and the PhD degree in cryptography from Xidian University, Xi'an, China, in 2007. He is a full professor with the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. He has been honored as a "Wan-Jiang Scholar" of Anhui Province since November 2013. His research interests include data security and privacy. He has published more than 60 technical papers at premium international journals and conferences, like the *IEEE Transactions on Intelligent Transportation Systems*, the *IEEE Transactions on Vehicular Technology*, *IEEE Global Communications Conference* and *IEEE Wireless Communications and Networking Conference*. He has served as a TPC member of many IEEE conferences, such as IEEE ICC, IEEE HPCC, IEEE TrustCOM. He is an associate editor of *Security and Communication Networks*, a member of the ACM, and a senior member of Chinese Computer Federation.

He is on the editorial board of *Security and Communication Networks*, *Computing and Informatics*, and the *International Journal of Embedded Systems*.



Xiaofeng Chen received the BS and MS degrees in mathematics from Northwest University, China, in 1998 and 2000, respectively, and the PhD degree in cryptography from Xidian University, in 2003. He is currently a professor with Xidian University. He has published more than 100 research papers in refereed international conferences and journals. His research interests include applied cryptography and cloud computing security. He has served as program/general chair or program committee member in more than 30 international conferences.

He is on the editorial board of *Security and Communication Networks*, *Computing and Informatics*, and the *International Journal of Embedded Systems*.



Jianfeng Ma received the PhD degree in computer software and communications engineering from Xidian University, Xi'an, China, in 1995. From 1999 to 2001, he was with Nanyang Technological University of Singapore as a research fellow. He is currently a professor with the School of Cyber Engineering, Xidian University, Xi'an, China. His current research interests focus on information and network security.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.